



PE FS25 SEMESTERARBEIT

Teil1: Dart Projekt in Unity

Teil2: Simulation eines Wagens auf einer
Rollbahn

Teil3: Simulation inelastischer Stöße und
Rotationen eines Wagens auf einer erweiterten
Rollbahn mit Luftkissentisch

Inhalt

1.	Dart Projekt in Unity - Einführung	3
1.1.	Überblick über die Implementierung.....	3
2.	OrbitCamera – Implementierung und Analyse	3
2.1.	OrbitCamera – Implementierung	3
2.2.	Skizzen zur Kamera-Bewegung	3
3.	Schwerkraft und Bewegung.....	5
3.1.	Implementierung der Gewichtskraft (rb.AddForce)	5
3.2.	Unterschied zwischen ForceMode.Force und ForceMode.Acceleration	5
3.3.	Warten auf Anweisungen	5
4.	Bewegungssteuerung des Dartpfeils.....	5
4.1.	Simulation der Flugbahn	6
4.2.	Implementierung der Startgeschwindigkeit (rb.linearVelocity)	6
4.3.	Einfluss des Koordinatensystems von Unity	6
5.	Analyse der Bewegungsdaten.....	6
5.1.	Analyse der Positionsdaten	7
5.2.	Analyse der Geschwindigkeitsdaten	8
5.3.	Analyse der Beschleunigungsdaten	9
6.	Anfang- und Endzustand der Simulation.....	10
7.	Simulation eines Wagens auf einer Rollbahn - Einführung	11
8.	Aufgabe 1 – Druckfedern	11
8.1.	Implementierung.....	11
8.2.	Analyse der Bewegung	12
8.3.	Plots und Auswertung.....	12
9.	Aufgabe 2 – Stossformeln.....	15
9.1.	Implementierung.....	15
9.2.	Plots und Auswertung.....	16
9.3.	Vergleich mit Feder-Modell	18
10.	Aufgabe 3 – Elastischer Stoss ohne Reibung	19
10.1.	Anpassung der Szene	19
10.2.	Geschwindigkeits- und Impulsanalyse	20
10.3.	Plots und Auswertung	20

11.	Aufgabe 4 – Elastischer Stoss mit Reibung	23
11.1.	Implementierung der Reibung	23
11.2.	Analyse von Impuls- und Energieerhaltung.....	23
11.3.	Plots und Auswertung	23
12.	Fazit zur Simulation eines Wagens auf einer Rollbahn	25
13.	Simulation inelastischer Stösse und Rotationen eines Wagens auf einer erweiterten Rollbahn mit Luftkissentisch – Einführung	28
14.	Energieerhaltung.....	28
14.1.	Physikalische Beschreibung	28
14.2.	Beschreibung der Implementierung.....	29
14.3.	Resultate und Interpretation	29
14.3.1.	Energieverlauf – ohne Reibung:	29
14.3.2.	Energieverlauf – mit Reibung:	30
14.3.3.	Fazit – Elastischer Stoss ohne und mit Reibung	31
15.	Zentraler inelastischer Stoss	32
15.1.	Physikalische Beschreibung	32
15.2.	Beschreibung der Implementierung.....	32
15.3.	Resultate und Interpretation	33
15.4.	Fazit – Zentraler inelastischer Stoss	36
16.	Inelastischer Stoss mit Rotation.....	37
16.1.	Physikalische Beschreibung	37
16.2.	Beschreibung der Implementierung.....	39
16.3.	Resultate und Interpretation	40
16.2.	Beschreibung der Implementierung.....	40
16.3.	Resultate und Interpretation	41
1.	Fazit zur Simulation inelastischer Stösse und Rotationen eines Wagens auf einer erweiterten Rollbahn mit Luftkissentisch	44
	Anhang	45

1. Dart Projekt in Unity - Einführung

In dieser Semesterarbeit wurde eine Simulation in Unity implementiert, in der ein Dartpfeil auf eine Dartscheibe zufliegt. Beide Objekte werden von der Schwerkraft beeinflusst und bewegen sich in Richtung des Bodens. Die Bewegung des Pfeils in Richtung der Scheibe bleibt jedoch konstant.

1.1. Überblick über die Implementierung

Das Projekt wurde in Unity umgesetzt, wobei C# zur Implementierung der Bewegung genutzt wurde. Der Code steuert die Kamera, die Bewegung der Objekte sowie den Export der physikalischen Daten zur späteren Analyse.

2. OrbitCamera – Implementierung und Analyse

Um die neue Position der Kamera zu bestimmen, betrachten wir die Transformation der Kamera relativ zur Dartscheibe unter Verwendung eines Rotationsquaternions und eines Positionsoffsets.

2.1. OrbitCamera – Implementierung

Das Skript OrbitCamera ist so implementiert, dass die Kamera einem festen Ziel folgt – in diesem Fall der Dartscheibe. Die Kamera bewegt sich in einer Kreisbahn um die Dartscheibe, während der Nutzer die Yaw- (Drehung um die Y-Achse) und Pitch-Werte (Neigung um die X-Achse) verändern kann. Dadurch kann der Blickwinkel auf die Szene flexibel angepasst werden.

Um die Kamera zu positionieren, wird die Drehung durch Quaternionen gesteuert. Der relevante Codeabschnitt lautet:

```
Quaternion rotation = Quaternion.Euler(pitch, yaw, 0);  
Vector3 offset = rotation * new Vector3(0, 0, distance);  
transform.position = targetInitialPosition + offset;
```

2.2. Skizzen zur Kamera-Bewegung

Zur Verdeutlichung der Funktionsweise der OrbitCamera wurden drei Skizzen erstellt, die die Drehbewegungen sowie die Position der Kamera relativ zur Dartscheibe veranschaulichen (siehe *Abbildung 1*):

1. Koordinatensystem mit Pitch & Yaw

Diese Skizze stellt das Koordinatensystem dar, in dem sich die Kamera bewegt. Sie zeigt die beiden relevanten Drehachsen:

- Yaw (rot) beschreibt die horizontale Drehbewegung der Kamera um die Y-Achse.
- Pitch (blau) beeinflusst die vertikale Neigung der Kamera entlang der X-Achse.

2. OrbitCamera Bewegung mit Pitch & Yaw

In dieser Darstellung wird die Bewegung der Kamera entlang einer Kreisbahn um die Dartscheibe gezeigt.

- Die rote Linie repräsentiert die Yaw-Drehung, durch die sich die Kamera auf einer Kreisbahn um die Dartscheibe bewegt.
- Die grüne Linie zeigt die Pitch-Drehung, welche die Höhe der Kamera verändert, während sie weiterhin auf die Dartscheibe ausgerichtet bleibt.

3. Kamera, Dartscheibe & Vektoren

Diese Skizze zeigt die Kamera als blauen Würfel in ihrer Position relativ zur Dartscheibe.

- Der Vektor von der Kamera zur Dartscheibe ist eingezeichnet, um zu verdeutlichen, dass sich die Kamera stets auf das Ziel ausrichtet.
- Dies ermöglicht eine dynamische Perspektivensteuerung, ohne dass die Kamera das Blickziel verliert.

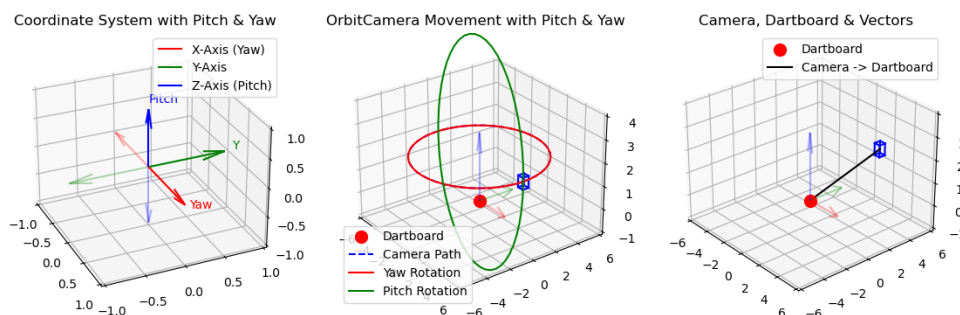


Abbildung 1: Skizzen zur Kamera-Bewegung mit Python

3. Schwerkraft und Bewegung

Die Schwerkraft wird durch den `Physics.gravity`-Vektor gesteuert, der standardmässig eine Beschleunigung von $\sim 9,81 \text{ m/s}^2$ in der y-Richtung bewirkt. Indem dieser Vektor in `FixedUpdate()` auf den `Rigidbody` des Pfeils und der Dartscheibe angewendet wird, fallen beide Objekte gleichmässig in Richtung Boden.

3.1. Implementierung der Gewichtskraft (`rb.AddForce`)

Um die Schwerkraft korrekt zu simulieren, wurde in den Controllern der beiden Objekte (`MoveDart.cs` und `MoveDartBoard.cs`) die Gewichtskraft mit `rb.AddForce()` implementiert. Dabei wurde zunächst ein eigener Gravitationsvektor definiert als auch der Unity-interne `Physics.gravity`-Vektor verwendet. Allerdings haben wir später beide Objekte auf den `Physics.gravity`-Vektor umgestellt, um die Einheitlichkeit sicherzustellen.

3.2. Unterschied zwischen `ForceMode.Force` und `ForceMode.Acceleration`

`ForceMode.Force` hängt von der Masse des Objektes ab, `ForceMode.Acceleration` hingegen nicht. Für die Schwerkraft ist `Acceleration` die korrekte Option (siehe auch [Galileo's Schiefer Turm von Pisa Experiment](#)).

3.3. Warten auf Anweisungen

Zu Beginn der Simulation sind beide Objekte mit `isKinematic = true` festgesetzt, sodass sie erst nach dem Drücken der Leertaste physikalisch beeinflusst werden. Dies stellt sicher, dass die Bewegung erst zum gewünschten Zeitpunkt startet und verbessert dadurch die Beobachtbarkeit des Experiments.

4. Bewegungssteuerung des Dartpfeils

Zu Beginn der Simulation soll sich der Dartpfeil mit einer konstanten Geschwindigkeit in Richtung der Dartscheibe bewegen. Dies wurde durch Setzen der `rb.linearVelocity` in der Methode `MoveDart.LaunchDart()` realisiert. Da Unity ein linkshändiges Koordinatensystem verwendet, wurde die Richtung der Bewegung entsprechend angepasst. Dadurch fliegt der Pfeil auf die Scheibe zu, während er gleichzeitig nach unten beschleunigt wird.

4.1. Simulation der Flugbahn

Die Flugbahn des Pfeils ist eine Kombination aus einer konstanten Vorwärtsbewegung und einer durch die Schwerkraft beeinflussten vertikalen Bewegung. In der y-Richtung wird der Pfeil durch die Gravitationskraft nach unten gezogen, wodurch er eine parabolische Flugbahn beschreibt.

Dies führt zu einer parabolischen Flugkurve, die durch die kinematische Gleichung für den schiefen Wurf beschrieben werden kann:

$$y(t) = y_0 + v_0 t + \frac{1}{2} g t^2$$

$$x(t) = x_0 + v_{0x} t$$

$$z(t) = z_0 + v_{0z} t$$

Die x- und z-Richtung sind nicht von der Schwerkraft betroffen und folgen einer gleichförmigen Bewegung mit konstanter Geschwindigkeit.

4.2. Implementierung der Startgeschwindigkeit (rb.linearVelocity)

Zu Beginn der Simulation soll sich der Dartpfeil mit einer konstanten Geschwindigkeit in Richtung der Dartscheibe bewegen. Dies wurde durch Setzen der rb.linearVelocity in der Methode MoveDart.LaunchDart() realisiert. Dabei ist zu beachten, dass rb.linearVelocity nicht verwendet werden soll, um die Geschwindigkeit während des Ablaufes zu ändern, da dies zu unrealistischem Verhalten führen würde.

4.3. Einfluss des Koordinatensystems von Unity

Da Unity ein linkshändiges Koordinatensystem verwendet, wurde die Richtung der Bewegung entsprechend angepasst. Dadurch fliegt der Pfeil unabhängig von der Schwerkraft in gerader Linie auf die Scheibe zu, während er gleichzeitig nach unten beschleunigt wird.

5. Analyse der Bewegungsdaten

Zur Untersuchung der Bewegungen von Pfeil und Scheibe wurden die gemessenen Positions-, Geschwindigkeits- und Beschleunigungswerte über die Zeit ausgewertet und grafisch dargestellt. Die nachfolgenden Abbildungen zeigen die Entwicklung der y- und z-Koordinaten sowie der entsprechenden Geschwindigkeiten und Beschleunigungen.

5.1. Analyse der Positionsdaten

In *Abbildung 2* ist die Position von Pfeil und Scheibe über die Zeit dargestellt. In der y-Richtung bewegen sich beide Objekte nach unten, was auf die Wirkung der Gravitation zurückzuführen ist. Die Kurven verlaufen fast identisch, was darauf hindeutet, dass beide Objekte einer ähnlichen Bewegung folgen.

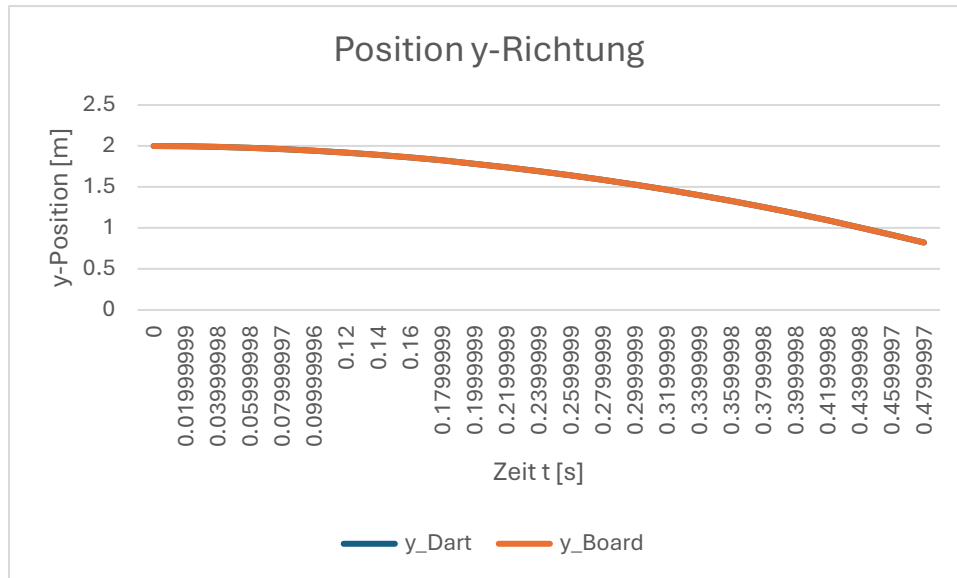


Abbildung 2: Diagramm zur Position in der y-Richtung

In der z-Richtung (siehe *Abbildung 3*) zeigt sich, dass der Pfeil zusätzlich eine Bewegungskomponente besitzt, während die Scheibe in dieser Richtung keine Bewegung aufweist. Dies bestätigt, dass sich die Scheibe ausschliesslich in der y-Richtung bewegt, während der Pfeil eine parabolische Flugbahn durchläuft.

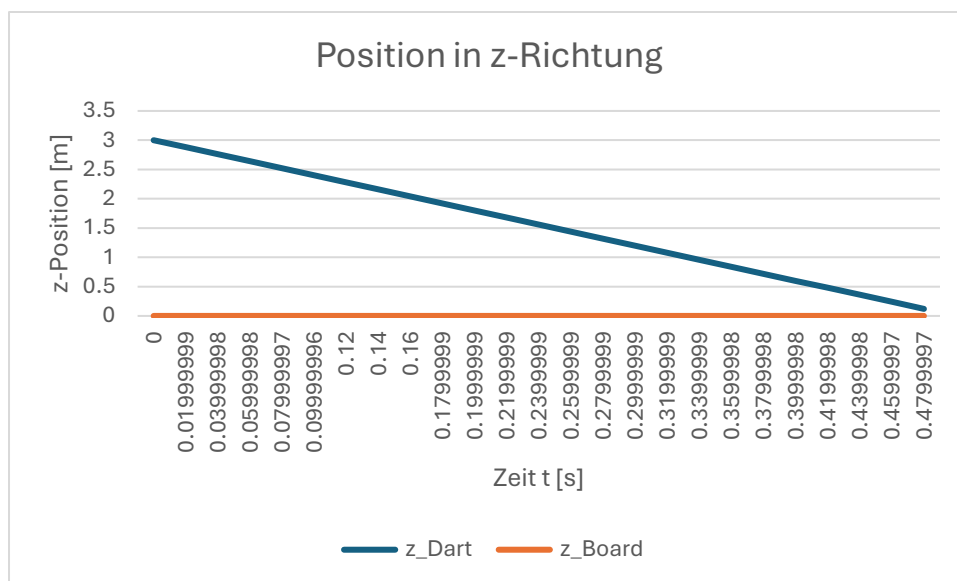


Abbildung 3: Diagramm zur Position in der z-Richtung

5.2. Analyse der Geschwindigkeitsdaten

Die Geschwindigkeit der Objekte ist in *Abbildung 4* für die y-Richtung und in *Abbildung 5* für die z-Richtung dargestellt. Die Geschwindigkeit in y-Richtung nimmt für beide Objekte stetig ab, was darauf hinweist, dass sie durch die Gravitation beschleunigt werden. Beide Kurven zeigen einen nahezu parallelen Verlauf, was darauf schliessen lässt, dass die Objekte mit der gleichen Beschleunigung fallen.

In der z-Richtung (siehe *Abbildung 5*) bleibt die Geschwindigkeit des Pfeils konstant, was darauf hindeutet, dass keine äussere Kraft in dieser Richtung wirkt. Die Scheibe bleibt bei $v_z = 0$, was bedeutet, dass sie in der z-Richtung keine Bewegung besitzt.

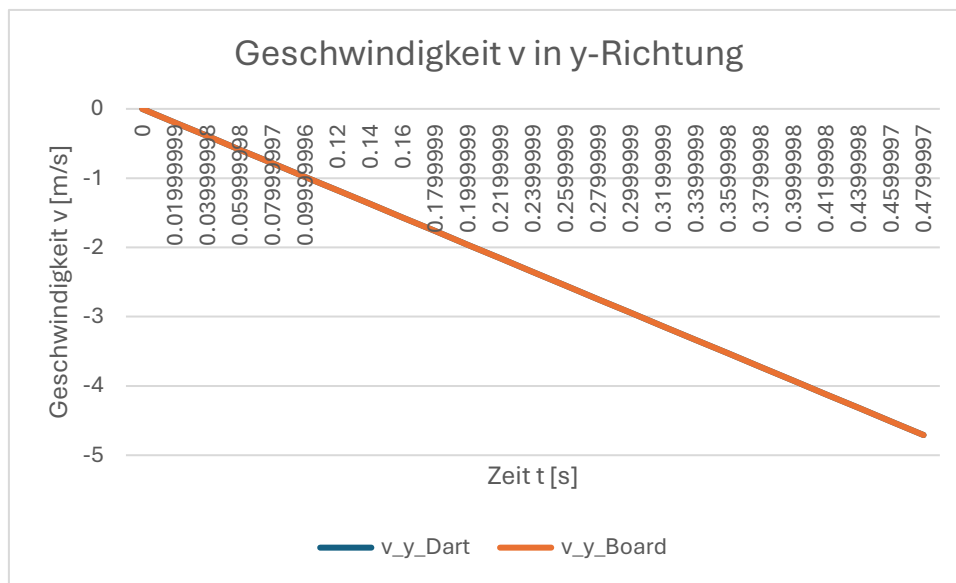


Abbildung 4: Diagramm zur Geschwindigkeit in der y-Richtung

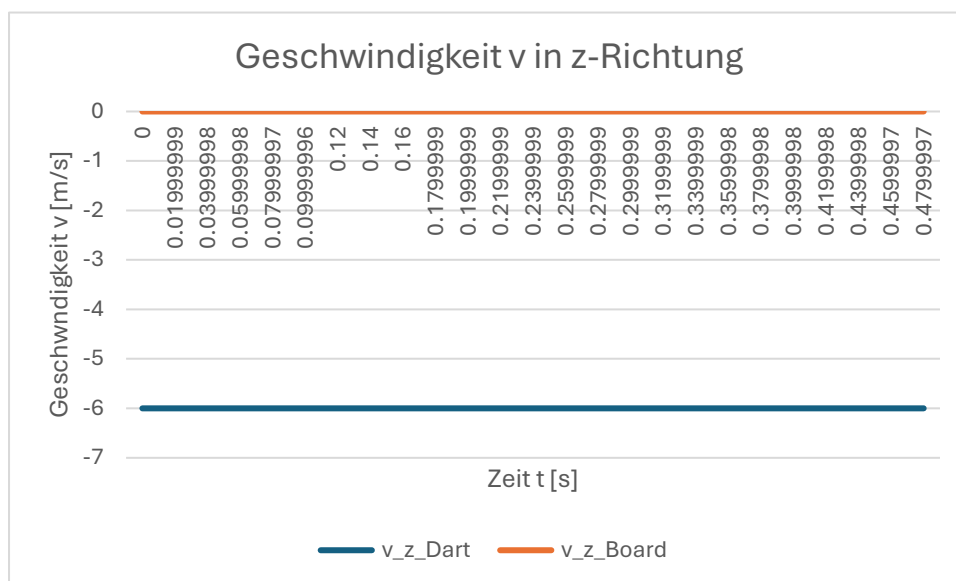


Abbildung 5: Diagramm zur Geschwindigkeit in der z-Richtung

5.3. Analyse der Beschleunigungsdaten

In *Abbildung 6* ist zu sehen, dass sowohl Dartscheibe wie auch der Dart selbst die gleiche senkrechte Beschleunigung aufweisen. Dies liegt daran, dass die Gravitation unabhängig von der Masse beschleunigt und in diesem Szenario kein Luftwiderstand modelliert wurde.

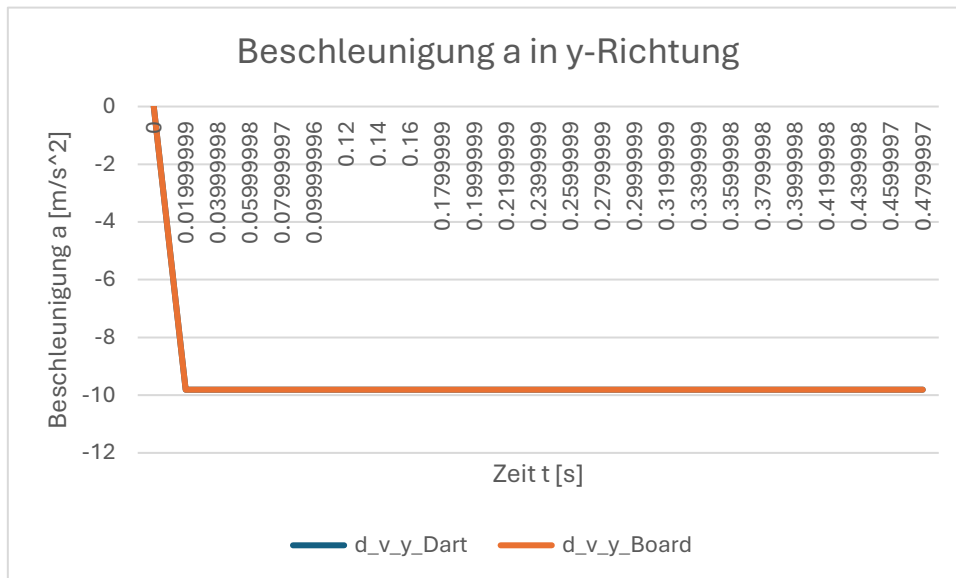


Abbildung 6: Diagramm zur Beschleunigung in der y-Richtung

6. Anfang- und Endzustand der Simulation

Abbildung 9 zeigt den Anfangszustand der Simulation, in dem sich der Dartpfeil in seiner Startposition befindet. Die Dartscheibe ist stationär, und die Szene ist bereit für den Simulationsstart.

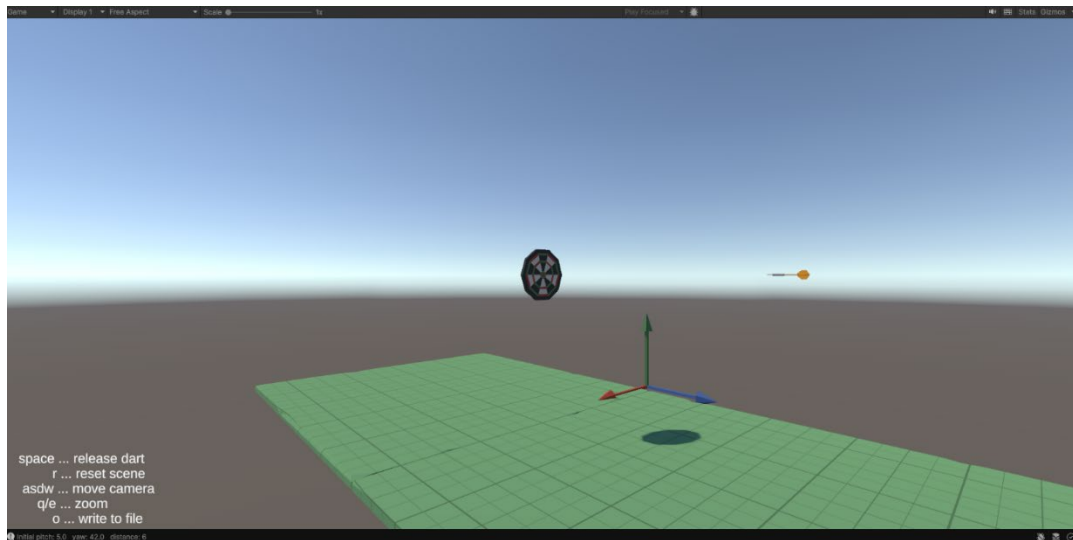


Abbildung 7: Anfangszustand

In Abbildung 10 ist der Endzustand zu sehen, nachdem der Pfeil seine Bewegung abgeschlossen hat.

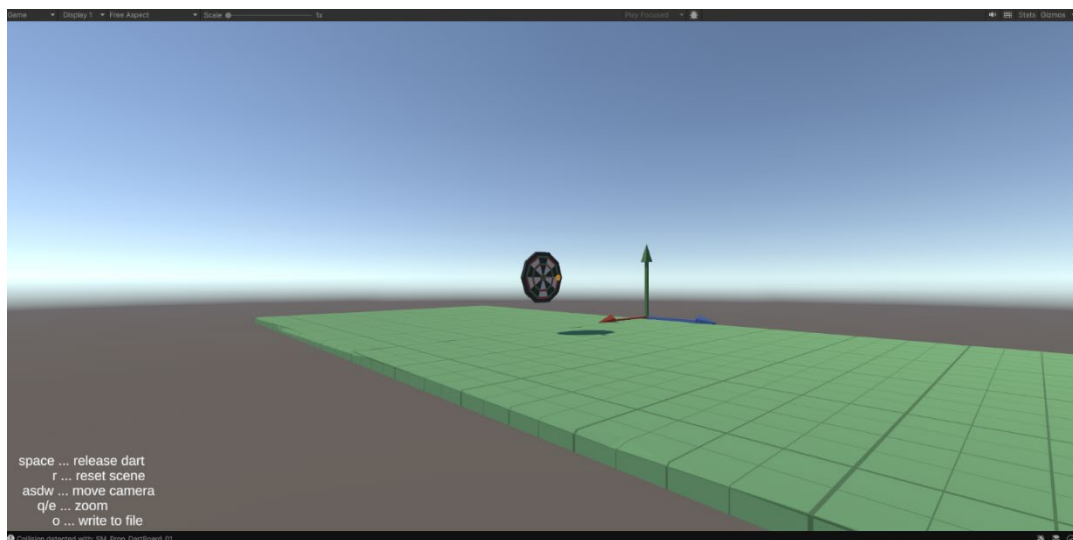


Abbildung 8: Endzustand

7. Simulation eines Wagens auf einer Rollbahn - Einführung

In diesem Bericht wird die Umsetzung und Analyse verschiedener physikalischer Szenarien in Unity beschrieben. Ziel ist es, die Bewegungen und Stösse eines Wagens auf einer Rollbahn mit Hilfe von Druckfedern, Stossformeln und Reibungskräften zu simulieren und auszuwerten.

8. Aufgabe 1 – Druckfedern

8.1. Implementierung

In der Ausgangsszene befindet sich ein Wagen mit einer Masse von 400 g, der reibungsfrei zwischen zwei Prellböcken gleiten kann. Ziel dieser Aufgabe war es, zwei virtuelle Druckfedern zu implementieren, die den Wagen bei Kontakt mit den Prellböcken zurückstossen und somit eine oszillierende Bewegung erzeugen.

Die Druckfedern wurden mit einer Ruhelänge von 15 cm und eine Federkonstante von 10 N/m parametrisiert. Sie wirken ausschliesslich dann, wenn der Abstand zwischen dem Wagen und dem jeweiligen Prellbock kleiner als die Ruhelänge ist – also wenn die Feder „komprimiert“ ist.

Die Kraftwirkung wurde mithilfe der Unity-Methode realisiert (Siehe Abbildung 9 und 10). Die Federn selbst sind nicht sichtbar und nur über die resultierenden Kräfte spürbar.

```
// === left spring
float deltaLeft = transform.position.z - leftBumper.position.z - carWidth / 2 - bumperWidth / 2;
if (deltaLeft > 0 && deltaLeft < springLength)
{
    compressionLeft = springLength - deltaLeft;
    forceLeft = springConstant * compressionLeft;
    rb.AddForce(new Vector3(0f, 0f, forceLeft));
}
```

Abbildung 9: linke Feder

```
// === right spring
float deltaRight = rightBumper.position.z - transform.position.z - carWidth / 2 - bumperWidth / 2;
if (deltaRight > 0 && deltaRight < springLength)
{
    compressionRight = springLength - deltaRight;
    forceRight = -springConstant * compressionRight;
    rb.AddForce(new Vector3(0f, 0f, forceRight));
}
```

Abbildung 10: rechte Feder

Die Implementierung basiert auf dem Hooke'schen Gesetz:

$$F_{Feder} = -k * (x - x_0)$$

wobei

- $k=10 \text{ N/m}$ die Federkonstante,
- x der aktuelle Abstand zwischen Wagen und Prellbock,
- und $x_0 = 0,15\text{m}$ die Ruhelänge ist.

Beide Federn wurden so implementiert, dass sie jeweils einseitig mit dem Wagen verbunden sind und ihre Kraft nur bei Kompression entfalten. Dadurch entsteht eine periodische Bewegung des Wagens zwischen den beiden Prellböcken.

8.2. Analyse der Bewegung

Die resultierende Bewegung des Wagens ist eine oszillierende, annähernd harmonische Schwingung. Ohne Energieverluste durch Reibung oder Dämpfung bleibt die Schwingungsamplitude konstant.

Die Bewegung lässt sich in drei Phasen unterteilen:

1. **Freie Bewegung:** Solange sich der Wagen zwischen den Prellböcken befindet und der Abstand zu beiden grösser als 15 cm ist, wirkt keine Federkraft.
2. **Kompression der linken/rechten Feder:** Sobald der Wagen einen der Prellböcke näher als 15 cm erreicht, wird er durch die entstehende Federkraft abgebremst und beschleunigt in die entgegengesetzte Richtung.
3. **Umkehrpunkt:** Der Wagen kommt kurzzeitig zum Stillstand, bevor er durch die Federkraft zurückgestossen wird.

8.3. Plots und Auswertung

Plot 1: Position – Zeit

Erwartung:

Da die Bewegung durch zwei idealisierte Druckfedern ohne Reibung beschrieben wird, ist eine **symmetrische Schwingung** zu erwarten. Die Auslenkung bleibt konstant, solange keine Energie verloren geht.

Beobachtung:

Die Position $z(t)$ zeigt eine periodische, annähernd harmonische Schwingung. Das Auto bewegt sich gleichmässig zwischen zwei Umkehrpunkten hin und her. (Siehe Abbildung 11)

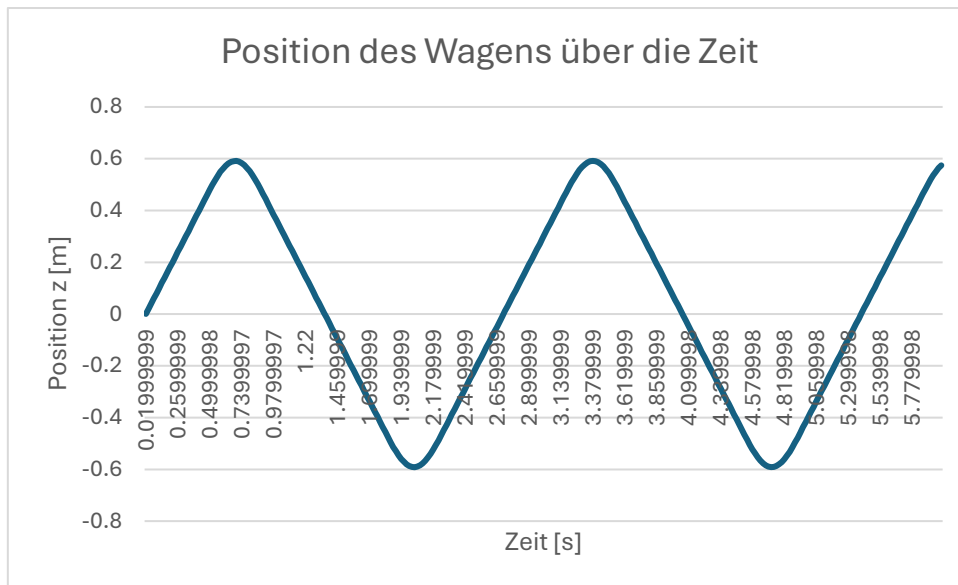


Abbildung 11: Plot Position – Zeit

Plot 2: Geschwindigkeit – Zeit

Erwartung:

Das entspricht dem Verhalten eines harmonischen Oszillators:

- Höchste Geschwindigkeit bei der Feder-Ruhelänge (in der Mitte)
- Geschwindigkeit = 0 an den Umkehrpunkten (maximale Kompression der Federn)

Beobachtung:

Die Geschwindigkeit wechselt periodisch zwischen positiv und negativ. Die Maxima der Geschwindigkeit treten in der Mitte der Strecke auf, die Nullstellen an den Umkehrpunkten. (Siehe Abbildung 12)

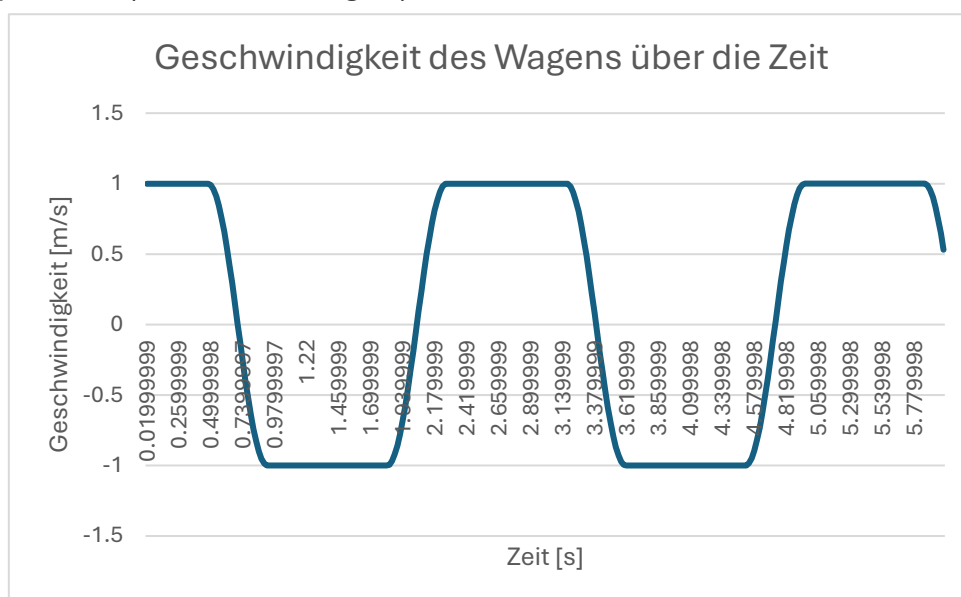


Abbildung 12: Plot Geschwindigkeit - Zeit

Plot 3: Beschleunigung – Zeit

Erwartung:

Die Beschleunigung ist proportional zur Federkraft (Hooke'sches). Sie ist daher am grössten, wenn die Feder maximal komprimiert ist – also am Umkehrpunkt der Bewegung.

Beobachtung:

Die Beschleunigung ist am stärksten an den Umkehrpunkten und geht in der Mitte gegen null. Es zeigt sich ein klarer Richtungswechsel bei jedem Stoss gegen die Feder. (Siehe Abbildung 13)

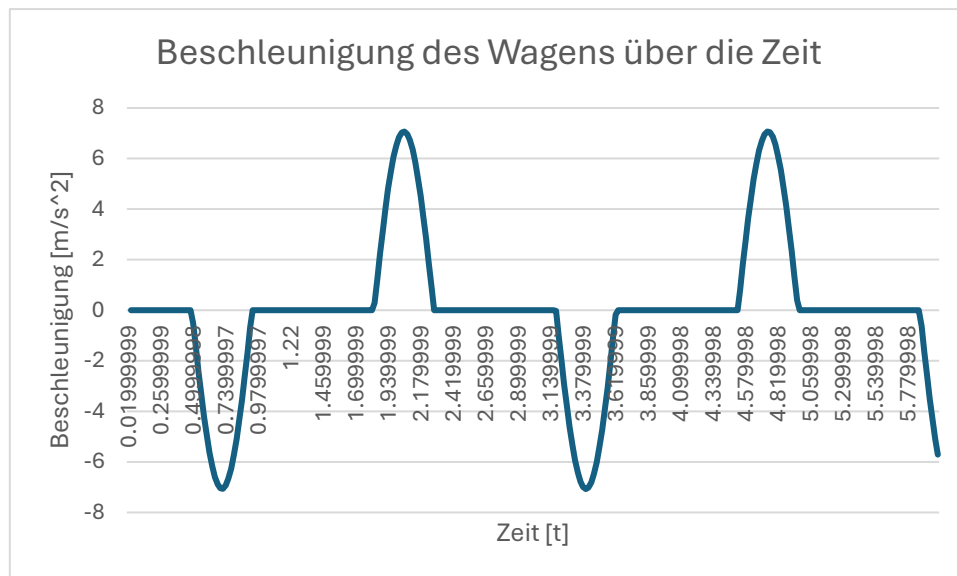


Abbildung 13: Plot Beschleunigung - Zeit

Plot 4: Impuls – Zeit

Erwartung:

Da keine äusseren Kräfte (wie Reibung) wirken, ist der Impuls des Wagens in jedem Halbschwingungsabschnitt erhalten. Der Wechsel des Vorzeichens zeigt die Richtungsänderung.

In einem geschlossenen System ohne Energieverlust bleibt der Impuls erhalten – genau das beobachten wir.

Beobachtung:

Der Impuls verläuft periodisch. In der Mitte ist der Betrag des Impulses maximal, an den Umkehrpunkten geht er gegen null. Der Gesamtverlauf ist symmetrisch. (Siehe Abbildung 14)

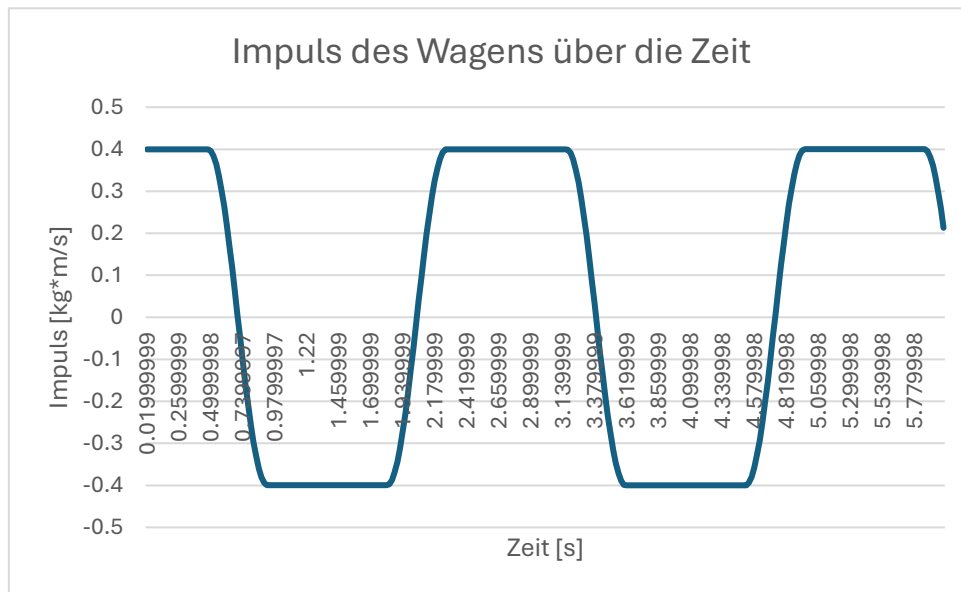


Abbildung 14: Plot Impuls-Zeit

Fazit aus den Plots

Die Bewegung erfüllt die Anforderungen an eine schwingende Bewegung mit idealen Druckfedern. Der Energieerhaltungssatz wird erfüllt, da keine Energieverluste durch Reibung oder Dämpfung modelliert wurden. Der Impuls des Wagens bleibt innerhalb des abgeschlossenen Systems erhalten, was ebenfalls der theoretischen Erwartung entspricht.

9. Aufgabe 2 – Stossformeln

9.1. Implementierung

In dieser Aufgabe wurde die rechte Feder entfernt und durch eine idealisierte elastische Kollision ersetzt. Sobald der Wagen den rechten Bumper erreicht, wird die neue Geschwindigkeit des Wagens basierend auf den Stossformeln berechnet. Da der Bumper fest verankert ist und somit als unbeweglich angenommen wird, vereinfacht sich die Formel zur Berechnung der Rückstossgeschwindigkeit:

$$v_{neu} = -v_{alt}$$

Die Richtung der Geschwindigkeit wird dadurch umgekehrt, der Betrag bleibt erhalten. Die Umsetzung erfolgte in Unity durch eine direkte Invertierung der z-Komponente der Geschwindigkeit, sobald die Position des Wagens kleiner oder gleich der Position des rechten Bumpers wurde. Eine Kraft wurde hier nicht angewendet, stattdessen wurde der Impuls direkt angepasst.

9.2. Plots und Auswertung

Plot 1: Position – Zeit

Erwartung:

Bei einem idealen elastischen Stoss wird der Richtungswechsel sofort und vollständig ausgeführt. Dies führt zu nicht-glatte, stückweise linearen Verläufen – genau wie hier beobachtet.

Beobachtung:

Die Position zeigt eine Bewegung zwischen zwei Grenzpunkten mit deutlichen Knickstellen an den Stosspunkten. Das Auto kehrt seine Bewegungsrichtung abrupt um. (Siehe Abbildung 15)

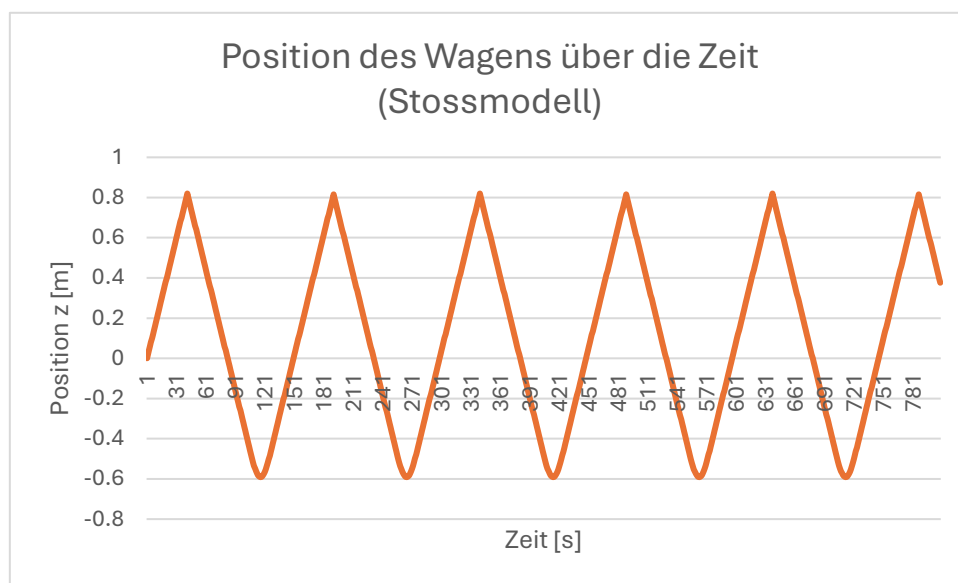


Abbildung 15: Plot Position-Zeit

Plot 2: Geschwindigkeit – Zeit

Erwartung:

In einem idealen Stossmodell mit festem Bumper ändert sich die Geschwindigkeit nach der Formel $v_{neu} = -v_{alt}$. Die beobachtete Sprungstelle spiegelt diese Richtungsumkehr exakt wider.

Beobachtung:

Die Geschwindigkeit zeigt **Sprungstellen** an den Stosspunkten: Der Betrag bleibt konstant, das Vorzeichen wechselt schlagartig. (Siehe Abbildung 8)

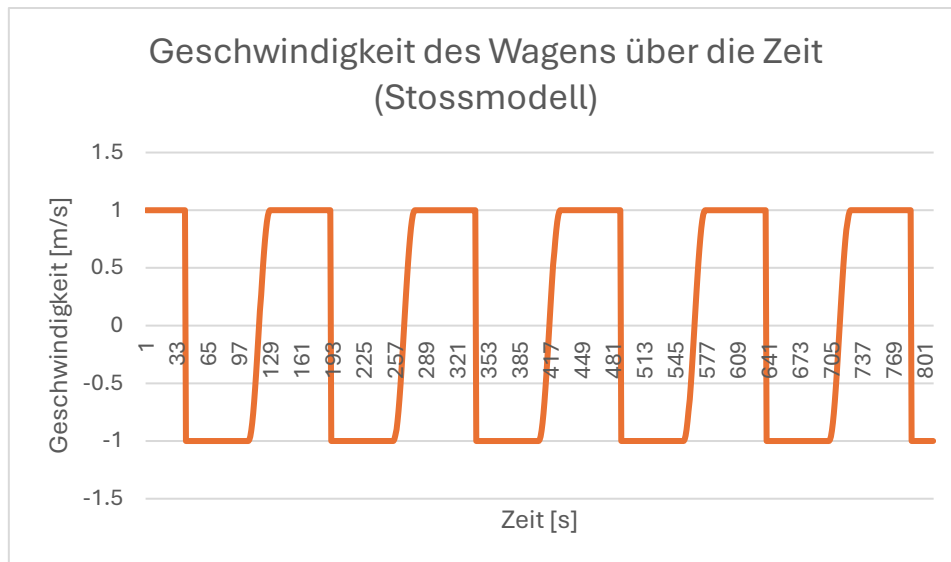


Abbildung 8: Plot Geschwindigkeit-Zeit

Plot 3: Beschleunigung – Zeit

Erwartung:

Da die Geschwindigkeitsänderung sofort erfolgt, resultiert daraus eine sehr grosse Beschleunigung über einen nahezu unendlich kleinen Zeitraum. Dies ist typisch für ein Modell ohne elastische Verformung oder Kontaktzeit.

Beobachtung:

Die Beschleunigung zeigt extrem scharfe Spitzen bei jedem Stosszeitpunkt. (Siehe Abbildung 16)

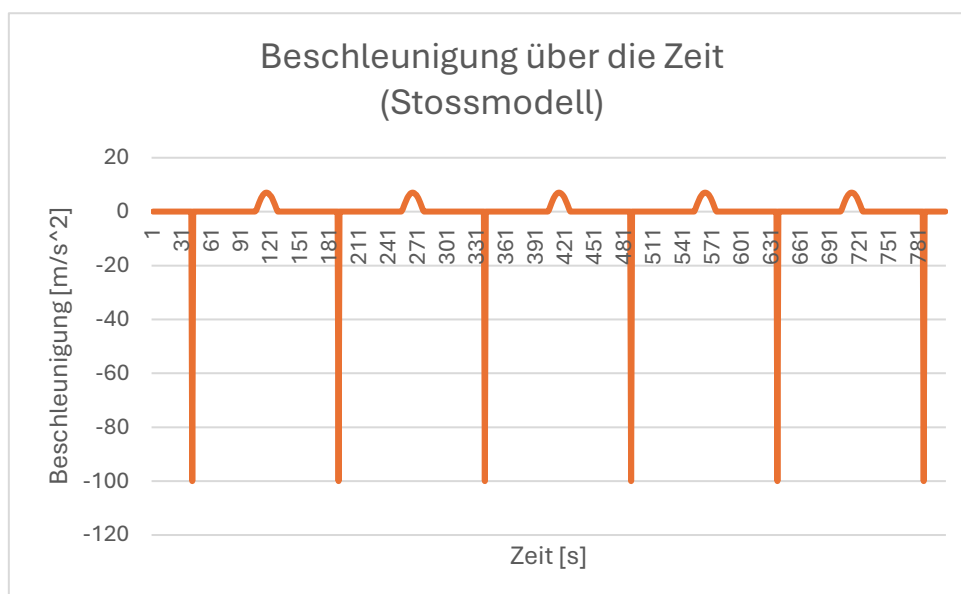


Abbildung 16: Plot Beschleunigung-Zeit

Plot 4: Impuls – Zeit

Erwartung:

Da der Stoss als vollständig elastisch und der Bumper als unbeweglich angenommen wird, bleibt der Impuls des Wagens betragsmässig erhalten. Der Richtungswechsel zeigt sich hier als Vorzeichenumkehr im Impulsverlauf.

Beobachtung:

Der Impuls des Wagens bleibt im Betrag konstant, wechselt jedoch an jedem Stoss das Vorzeichen. Die Sprungstellen stimmen exakt mit den Stosszeitpunkten überein. (Siehe Abbildung 17)

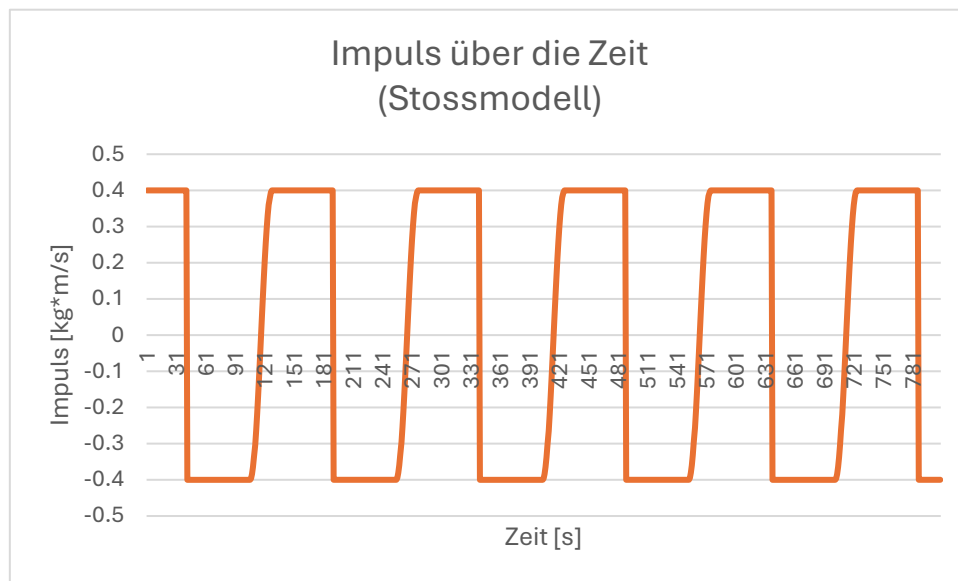


Abbildung 17: Plot Impuls-Zeit

Fazit aus den Plots

Die Plots bestätigen das physikalische Verhalten eines idealisierten elastischen Stosses mit einem festen Bumper. Die Position und Geschwindigkeit des Wagens zeigen abrupte Richtungswechsel, wie sie bei einer stossbasierten Bewegungsumkehr zu erwarten sind. Besonders deutlich wird dies durch die Sprungstellen in der Geschwindigkeit und die Knickpunkte in der Position.

Die Beschleunigung weist extreme Spitzen auf, da sich die Geschwindigkeit rechnerisch innerhalb eines einzelnen Zeitschritts umkehrt. Dies ist eine typische Folge idealisierter Stossmodelle ohne Verformung oder Kontaktzeit. Der Impulsverlauf bestätigt die Erhaltung des Impulsbetrags, wobei das Vorzeichen entsprechend der Bewegungsrichtung wechselt.

9.3. Vergleich mit Feder-Modell

Im Gegensatz zum Feder-Modell, bei dem die Rückstellkraft kontinuierlich wirkt und über eine gewisse Strecke hinweg aufgebaut wird, ist der Stoss hier als punktuell

Ereignis modelliert. Das bedeutet, dass sich die Bewegungsumkehr unmittelbar und ohne „sichtbare“ Verzögerung vollzieht.

Während das Feder-Modell zu einer harmonischen Schwingung mit stetigem Übergang führt, zeigt der Stoss eine harte Richtungsänderung. Dies wirkt sich auch auf die Plots der Beschleunigung und Kraft aus: Beim Stoss ergibt sich ein abrupter Sprung, während die Feder eine glattere Kraftkurve erzeugt.

Beide Modelle erhalten den Impuls, unterscheiden sich jedoch deutlich im zeitlichen Verlauf der Kräfte und Bewegungen.

10. Aufgabe 3 – Elastischer Stoss ohne Reibung

10.1. Anpassung der Szene

Für diese Aufgabe wurde der linke Bumper so modifiziert, dass er nicht mehr fest im Boden verankert ist, sondern sich reibungsfrei auf der Bahn bewegen kann. Dadurch kommt es beim Kontakt mit dem Wagen zu einem elastischen Stoss zwischen zwei beweglichen Körpern: dem Wagen (Masse $m_1=0,4$ kg) und dem Bumper (Masse $m_2=1,0$ kg). Der rechte Bumper bleibt weiterhin fixiert, um eine periodische Bewegung zu ermöglichen.

Die Umsetzung in Unity erfolgte, indem die Constraints des linken Bumpers so angepasst wurden, dass nur noch Bewegung entlang der z-Achse möglich ist. Reibung wurde vollständig deaktiviert. Der Stoss zwischen Wagen und Bumper wird durch die in Unity auftretende Kontaktphysik realisiert. Zusätzlich wurde überprüft, ob die resultierenden Geschwindigkeiten dem elastischen Stossgesetz entsprechen. (Siehe Abbildung 18)

```
float deltaLeft = transform.position.z - leftBumper.position.z - carWidth / 2 - bumperWidth / 2;
if (deltaLeft <= 0) // Collision detected
{
    // Get masses and velocities
    float m1 = rb.mass; // Mass of the car
    float m2 = leftBumper.mass; // Mass of the left bumper
    float v1 = rb.linearVelocity.z; // Velocity of the car
    float v2 = leftBumper.linearVelocity.z; // Velocity of the left bumper

    // Calculate new velocities after elastic collision
    float v1New = ((m1 - m2) / (m1 + m2)) * v1;
    float v2New = ((2 * m1) / (m1 + m2)) * v1;

    // Apply new velocities
    rb.linearVelocity = new Vector3(0f, 0f, v1New);
    leftBumper.linearVelocity = new Vector3(0f, 0f, v2New);

    Debug.Log("Elastischer Stoß am linken Prellbock");
}
```

Abbildung 18: Code vom elastischen Stoss

10.2. Geschwindigkeits- und Impulsanalyse

Nach den Stossformeln für einen elastischen Stoss zweier Körper ergibt sich:

$$v1' = \frac{m1 - m2}{m1 + m2} \cdot v1 \quad v2' = \frac{2m1}{m1 + m2} \cdot v1$$

$$v1' = \frac{m1 - m2}{m1 + m2} \cdot v1 = \frac{0.4kg - 1kg}{0.4kg + 1kg} \cdot v1 = -0.428 \cdot v1$$

$$v2' = \frac{2m1}{m1 + m2} \cdot v1 = \frac{2 \cdot 0.4kg}{0.4kg + 1kg} \cdot v1 = 0.571 \cdot v1$$

Der Wagen verliert durch den Stoss seine Geschwindigkeit und fährt in die entgegengesetzte Richtung, während der Bumper proportional beschleunigt wird und auch in die entgegengesetzte Richtung vom Wagen über die Bahn gleitet.

Der Impuls beider Körper bleibt im gesamten Stossvorgang erhalten. Der Gesamtimpuls ist vor und nach dem Stoss gleich, nur die Verteilung auf die beiden Körper verändert sich.

$$p_{total} = m1 \cdot v1 + m2 \cdot v2 = m1 \cdot v1' + m2 \cdot v2'$$

10.3. Plots und Auswertung

Plot 1: Geschwindigkeiten der beteiligten Körper

Erwartung:

Für einen idealen, elastischen Stoss ohne Reibung zwischen zwei Körpern ergibt sich aus der Stossformel eine sofortige und impulsbasierte Änderung der Geschwindigkeiten. Die Geschwindigkeit wird je nach Massenverhältnis auf beide Körper aufgeteilt. Ist der Prellbock schwerer, übernimmt er einen kleineren Teil der Geschwindigkeit; bei annähernd gleichen Massen tauschen die Körper ihre Bewegungen fast vollständig. Dieses Verhalten wurde in der Simulation korrekt wiedergegeben.

Erwartung:

Für einen idealen, elastischen Stoss ohne Reibung zwischen zwei Körpern ergibt sich aus der Stossformel eine sofortige und impulsbasierte Änderung der Geschwindigkeiten. Die Geschwindigkeit wird je nach Massenverhältnis auf beide Körper aufgeteilt. Ist der Prellbock schwerer, übernimmt er einen kleineren Teil der Geschwindigkeit; bei annähernd gleichen Massen tauschen die Körper ihre Bewegungen fast vollständig.

Beobachtung:

Zu Beginn bewegt sich nur der Wagen mit positiver Geschwindigkeit auf den Prellbock zu. Beim Stoss ändert sich seine Geschwindigkeit schlagartig – der Wagen bewegt sich leicht rückwärts. Gleichzeitig beginnt sich der Prellbock plötzlich mit konstanter Geschwindigkeit in Bewegungsrichtung des Wagens zu bewegen. Diese Sprungstellen sind in den Plots klar sichtbar. (Siehe Abbildung 19)

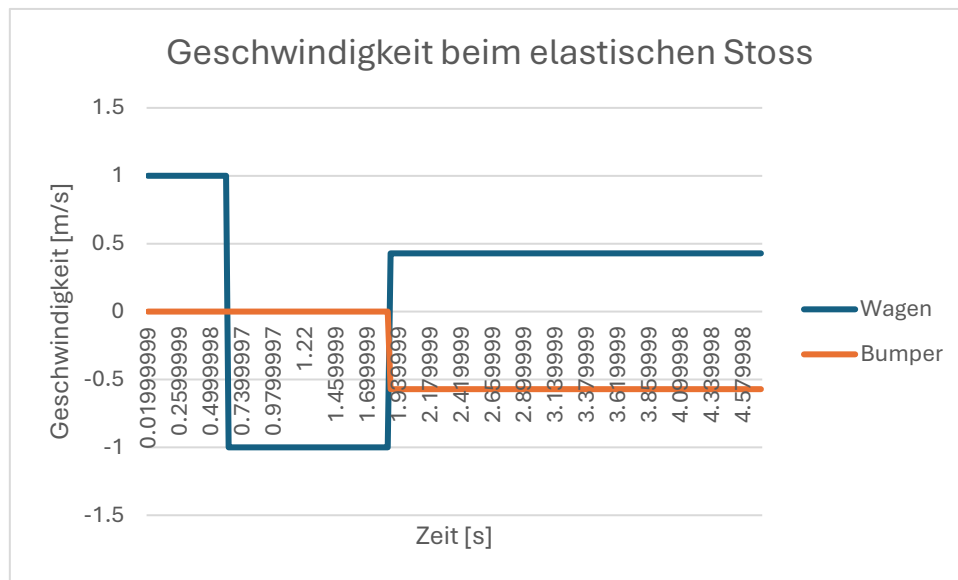


Abbildung 19: Plot Geschwindigkeit-Zeit

Plot 2: Impulse der beteiligten Körper**Erwartung:**

Da es sich um ein abgeschlossenes System ohne äussere Kräfte handelt und der Stoss elastisch ist, muss der Gesamtimpuls erhalten bleiben.

Beobachtung:

Der Impuls des Wagens sinkt beim Stoss abrupt ab, während der Prellbock in gleichem Masse Impuls aufnimmt. Der Gesamtimpuls des Systems bleibt während des gesamten Vorgangs konstant, was sich im Plot durch eine flache Linie darstellt. (Siehe Abbildung 20)

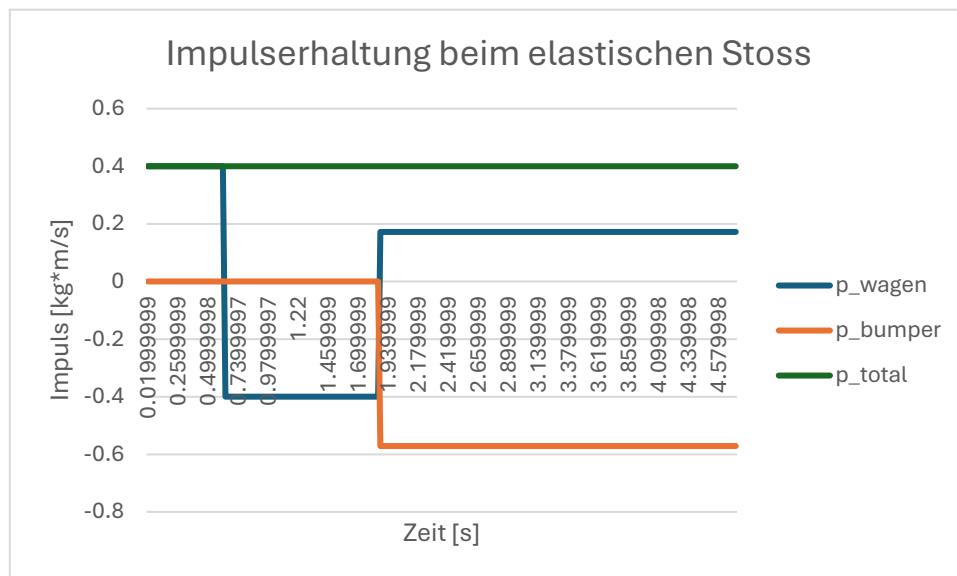


Abbildung 20: Plot Impuls-Zeit

Fazit aus den Plots:

Die Auswertung der Geschwindigkeits- und Impulsverläufe zeigt, dass Unity den elastischen Stoss zwischen dem Wagen und dem beweglichen Prellbock korrekt abbildet. Vor dem Stoss bewegt sich ausschliesslich der Wagen mit positiver Geschwindigkeit auf den Bumper zu, während dieser ruht. Beim Kontakt kommt es zu einem plötzlichen Geschwindigkeitswechsel: Der Wagen wird leicht zurückgestossen, während der Bumper beginnt, sich mit einer konstanten Geschwindigkeit in Bewegungsrichtung des Wagens zu bewegen. Die beobachteten Sprungstellen in den Geschwindigkeitsverläufen entsprechen exakt dem Verhalten eines idealen elastischen Stosses, wie es aus der Theorie bekannt ist.

Auch der Impulsverlauf bestätigt die physikalischen Erwartungen. Während der Wagen vor dem Stoss den gesamten Impuls des Systems trägt, geht dieser beim Stoss vollständig auf den Prellbock über. Die Summe der Impulse beider Körper bleibt über die gesamte Zeit konstant. Dies belegt, dass der Gesamtimpuls des Systems erhalten bleibt und keine externen Kräfte oder Reibungseinflüsse wirken. Insgesamt zeigen die Plots, dass die Simulation die theoretischen Grundlagen des elastischen Stosses zuverlässig abbildet. Der abrupte Impuls- und Geschwindigkeitsaustausch sowie die Konstanz des Gesamtimpulses sind in den Daten deutlich nachvollziehbar.

11. Aufgabe 4 – Elastischer Stoss mit Reibung

11.1. Implementierung der Reibung

Für diese Aufgabe wurde der linke Prellbock so konfiguriert, dass er sich reibungsbehaftet entlang der z-Achse bewegen kann. Ziel war es, das physikalische Verhalten eines realen Systems mit Energieverlust zu simulieren. Die Reibung wurde als laminare viskose Dämpfung modelliert, wobei die Reibungskraft proportional zur Geschwindigkeit des Bumpers wirkt:

$$F_{Reib} = -k * v_{Bumper}$$

Die Implementierung erfolgte durch Anwendung einer kontinuierlichen Kraft auf den Rigidbody des Bumpers in der Methode FixedUpdate():

```
// == friction force on the left bumper
Vector3 frictionForce = -frictionCoefficient * leftBumper.linearVelocity.normalized * leftBumper.linearVelocity.magnitude;
leftBumper.AddForce(frictionForce);
```

Der Reibungskoeffizient k wurde so gewählt, dass sich der Bumper nach zwei bis drei Stössen merklich abbremst und anschliessend zum Stillstand kommt.

11.2. Analyse von Impuls- und Energieerhaltung

Impuls:

Der Impuls des Systems wird durch die Reibungskraft nicht mehr erhalten bleiben. Die Reibung wirkt als äussere, nicht-konservative Kraft und führt dazu, dass mit jeder Wechselwirkung ein Teil des Impulses „verloren geht“. Dies zeigt sich im Gesamtimpuls, der nach jedem Stoss abnimmt.

Energie:

Auch die kinetische Energie des Systems wird durch die Reibung reduziert. Während im reibungsfreien Fall die Energie zwischen Wagen und Bumper hin- und herwandert, wird sie in diesem Fall durch Reibungsarbeit kontinuierlich dissipiert. Dies führt dazu, dass sich beide Körper zunehmend verlangsamen und schliesslich zur Ruhe kommen.

Die Simulation bildet damit die physikalisch korrekte Wirkung eines Dämpfungsmechanismus ab: Energie und Impuls sind nicht erhalten, sondern werden über die Reibung in andere Energieformen (z. B. Wärme) überführt.

11.3. Plots und Auswertung

Plot 1: Impuls - Zeit

Erwartung:

In einem System mit Reibung ist zu erwarten, dass der Impuls nicht erhalten bleibt, da Reibung eine äussere, nicht-konservative Kraft darstellt. Sie übt über die Zeit hinweg eine Gegenkraft auf den Bumper aus und entzieht dem System damit Impuls. Dieser

Verlauf passt zu einem realen physikalischen System, in dem Stossenergie über Reibung in Wärme übergeht.

Beobachtung:

Der Plot zeigt deutlich, dass sowohl der Impuls des Wagens als auch des Bumpers nach jedem Stoss abnimmt. Der Wagen verliert bei jedem Stoss an Impuls, während der Bumper diesen zunächst teilweise aufnimmt, aber durch die Reibung schnell abbremst. Die Summe der beiden Impulse – der Gesamtimpuls des Systems – sinkt stetig über die Zeit. Dies deutet darauf hin, dass der Impuls nicht im System verbleibt, sondern durch Reibung allmählich abgebaut wird. (Siehe Abbildung 21)

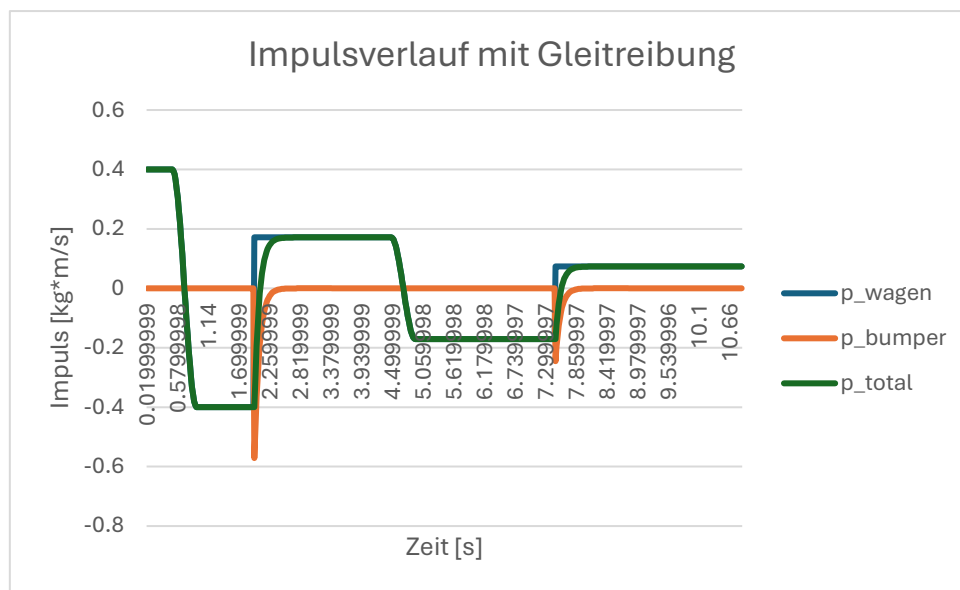


Abbildung 21: Plot Impuls-Zeit

Plot 2: kinetische Energie – Zeit

Erwartung:

Da Reibung vorhanden ist, ist die mechanische Energie nicht mehr erhalten. Ein Teil der kinetischen Energie wird in Reibungsarbeit umgesetzt und geht dadurch für das mechanische System verloren. Der kontinuierliche Energieverlust entspricht dem Verhalten eines realen, gedämpften Systems, wie man es z. B. bei Stossdämpfern oder Materialreibung beobachten kann.

Beobachtung:

Die kinetische Energie des Wagens nimmt mit jedem Stoss ab, ebenso wie jene des Bumpers. Besonders auffällig ist, dass die Gesamtenergie des Systems kontinuierlich sinkt. Zu Beginn der Simulation ist der Wagen alleiniger Energieträger; nach dem Stoss verteilt sich die Energie auf beide Körper, nimmt jedoch insgesamt ab, bis das System schliesslich vollständig zur Ruhe kommt. (Siehe Abbildung 22)

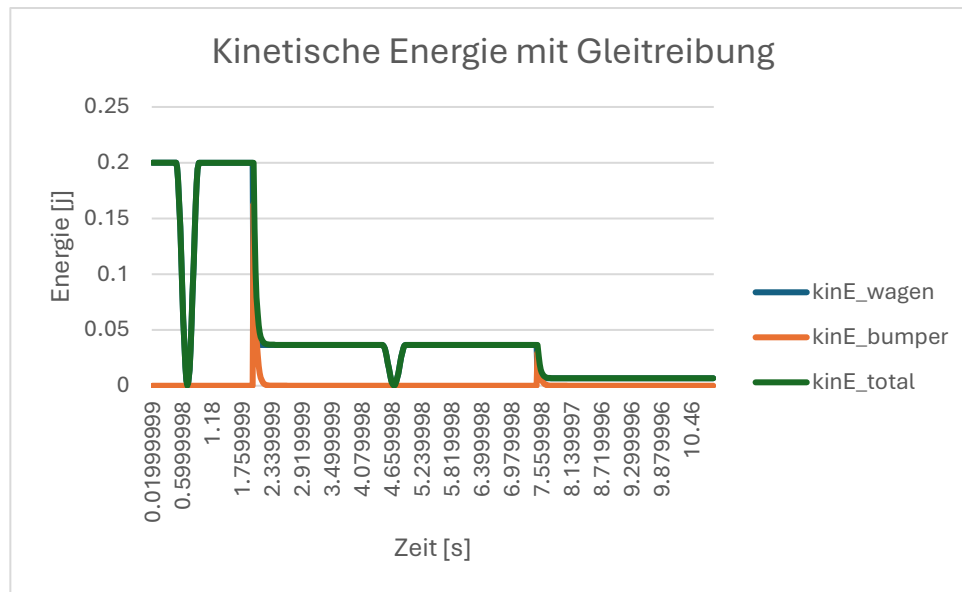


Abbildung 22: Plot Energieerhaltung

Fazit aus den Plots:

Die dargestellten Plots zeigen die physikalischen Auswirkungen der eingeführten Reibungskraft. Im Gegensatz zu den vorherigen Aufgaben, bei denen Impuls und Energie erhalten blieben, führt die Reibung nun zu einem stetigen Verlust beider Größen. Die Impulse der beteiligten Körper nehmen nach jedem Stoss ab, und der Gesamtimpuls des Systems sinkt kontinuierlich.

Auch die kinetische Energie des Systems folgt diesem Trend. Sowohl Wagen als auch Prellbock verlieren bei jeder Wechselwirkung einen Teil ihrer Bewegungsenergie, bis das System schliesslich zum Stillstand kommt. Die kontinuierlich abnehmende Gesamtenergie bestätigt, dass Reibung als dissipativer Prozess wirkt und mechanische Energie in nicht beobachtbare Formen (z. B. Wärme) umwandelt.

Die Plots zeigen somit, dass die implementierte Reibung nicht nur technisch korrekt wirkt, sondern auch ein realitätsnahes Verhalten erzeugt: ein abklingendes System, das sich mit der Zeit vollständig beruhigt.

12. Fazit zur Simulation eines Wagens auf einer Rollbahn

In **Aufgabe 1** wurde ein System mit zwei idealen Druckfedern implementiert. Die Simulation zeigte eine symmetrische, harmonische Schwingung, wie sie für ein reibungsfreies System mit idealen Rückstellkräften typisch ist. Sowohl die Energie- als auch die Impulserhaltung waren vollständig gegeben.

In **Aufgabe 2** wurde die rechte Feder durch ein ideales Stossmodell ersetzt. Die plötzliche Richtungsumkehr des Wagens wurde korrekt umgesetzt und zeigte sich in den Sprungstellen der Geschwindigkeit. Im Vergleich zum kontinuierlichen Kraftverlauf

der Feder war das Verhalten des Stossmodells diskret und kantig – jedoch physikalisch korrekt.

Aufgabe 3 erweiterte das Modell um einen beweglichen Prellbock, was einen echten elastischen Stoss zwischen zwei Körpern ermöglichte. Die Geschwindigkeitsteilung und Impulsübertragung stimmten mit den theoretischen Stossformeln überein, und der Gesamtimpuls des Systems blieb erhalten. Die Simulation veranschaulichte sehr gut den Unterschied zum feststehenden Bumper.

In **Aufgabe 4** wurde eine Reibungskraft eingeführt, die das System realitätsnäher machte. Es zeigte sich, dass durch die Dämpfung sowohl Impuls als auch Energie schrittweise aus dem System verschwanden, bis schliesslich vollständiger Stillstand eintrat. Die Bewegung wurde durch die Reibung nicht nur abgeschwächt, sondern vollständig abgebaut – ein typisches Verhalten für reale mechanische Systeme mit Energieverlusten.

Im Folgenden werden die vier simulierten Modelle hinsichtlich ihrer physikalischen Eigenschaften, der Modellierung in Unity und der Auswirkungen auf Impuls und Energie gegenübergestellt:

Modell	Beschreibung	Impuls- erhaltung	Energie- erhaltung	Bewegung
Doppelfeder (Aufgabe 1)	Zwei Druckfedern sorgen für harmonische Schwingung.			Periodisch
Stossformel (Aufgabe 2)	Rechte Feder entfernt, Stoss gegen festen Bumper mit $v' = -v$.			Kantig, stossartig
Beweglicher Bumper (Aufgabe 3)	Stoss mit elastischer Wechselwirkung zwischen Wagen und Bumper mit Masse.			Austauschbewegung
Stoss mit Reibung (Aufgabe 4)	Wie Aufgabe 3, jedoch mit Reibungskraft auf dem Bumper.			Abklingend

Diese Versuche zeigen auf, dass die Gesetze der Impuls- und Energieerhaltung nur in geschlossenen Systemen gültig sind. Sobald Reibung in das metaphorische Spiel kommt, gibt es eine Wechselwirkung mit Elementen ausserhalb des Systems, in diesem Fall der virtuelle Boden. Dadurch verkompliziert sich natürlich die ganze Berechnung, da man entweder keine perfekte Erhaltung mehr hat oder man viel mehr als eigentlich geplant simulieren muss.

Auch zeigt sich, dass sogar simple Modelle bei der Einführung kleiner Details grundlegend andere Resultate liefern können.

13. Simulation inelastischer Stöße und Rotationen eines Wagens auf einer erweiterten Rollbahn mit Luftkissentisch – Einführung

In Teil 3 der Semesterarbeit wird die bisher eindimensionale Bewegung des Wagens auf der Rollbahn in eine zweidimensionale Bewegung auf einem Luftkissentisch erweitert. Dadurch können die beteiligten Objekte nicht nur gleiten, sondern auch rotieren. Ziel ist es, verschiedene Stossszenarien – sowohl elastisch als auch inelastisch – unter realitätsnahen Bedingungen zu analysieren. Besonderes Augenmerk liegt dabei auf der Erhaltung von Energie und Impuls sowie auf der Betrachtung von Drehimpulsen bei nicht-zentralen Stößen. Die Simulationen werden mit der Unity-Physikengine umgesetzt und durch theoretische Berechnungen ergänzt und validiert.

14. Energieerhaltung

14.1. Physikalische Beschreibung

In dieser Aufgabe wird die Energieerhaltung bei elastischen Stößen zwischen einem Wagen und einem Prellbock untersucht. Dabei wurden zwei Szenarien verglichen:

1. Elastischer Stoss ohne Reibung:

Die Geschwindigkeit des Wagens wird beim Aufprall mithilfe der Stossformel angepasst. Die Simulation entspricht einem idealisierten, zentralen Stoss zwischen zwei Körpern auf einer reibungsfreien Fläche.

2. Elastischer Stoss mit Reibung:

Zusätzlich zur Stossformel wird eine Druckfeder eingebaut, die den Kontakt zwischen Wagen und Prellbock modelliert. Gleichzeitig wirkt eine Reibungskraft auf den Prellbock, wodurch Energie dissipiert wird.

In einem abgeschlossenen System ohne Reibung sollte die mechanische Gesamtenergie erhalten bleiben:

$$E_{\text{gesamt}} = E_{\text{kin,wagen}} + E_{\text{kin,bumper}} = \text{konstant}$$

Bei Einführung von Reibung und Feder kann ein Teil der Energie in Wärme oder Verformung übergehen. Es gilt dann:

$$E_{\text{nach dem Stoss,gesamt}} < E_{\text{vor dem Stoss,gesamt}}$$

14.2. Beschreibung der Implementierung

- **Stoss ohne Reibung:**

Der Wagen (0.4 kg) stösst elastisch mit einem Prellbock (1.0 kg) zusammen. Die neue Geschwindigkeit wird gemäss der Stossformel in einem C#-Skript berechnet:

$$v1' = \frac{m1 - m2}{m1 + m2} \cdot v1 \quad v2' = \frac{2m1}{m1 + m2} \cdot v1$$

- **Stoss mit Reibung:**

Eine Druckfeder (Hooke'sches Gesetz) modelliert die Stosswirkung, kombiniert mit einer Reibungskraft:

$$F_{Feder} = -k * \Delta x$$

$$F_{Reibung} = -c * v$$

Die Bewegung des Prellbocks wird durch Rigidbody-Physik in Unity berechnet.

14.3. Resultate und Interpretation

14.3.1. Energieverlauf – ohne Reibung:

Erwartung:

In einem idealisierten elastischen Stoss ohne Reibung auf einem Luftkissentisch sollte die Gesamtenergie des Systems konstant bleiben. Vor dem Stoss bewegt sich ausschliesslich der Wagen, seine gesamte kinetische Energie entspricht somit der Systemenergie. Während des Stosses wird diese Energie gemäss der Stossformel auf den Prellbock übertragen – abhängig vom Massenverhältnis. Die Erwartung lautet:

- **E_kin_wagen:** konstant vor dem Stoss, nimmt beim Stoss sprunghaft ab
- **E_kin_bumper:** zunächst null, nimmt beim Stoss sprunghaft zu
- **E_gesamt:** bleibt konstant über die gesamte Zeit

Da keine Feder und keine Reibung wirken, sollte es keinen Energieverlust geben, sondern lediglich eine Umverteilung der kinetischen Energie.

Beobachtung:

Der Plot (Siehe Abbildung) zeigt genau dieses Verhalten:

- Vor dem Stoss trägt der Wagen die gesamte Energie (ca. 0.2 J), der Prellbock ist unbewegt.
- Beim Stoss (ca. bei $t = 90$) sinkt die kinetische Energie des Wagens abrupt, während die des Prellbocks sprunghaft steigt – ein typisches Verhalten eines zentral-elastischen Stosses.

- Nach dem Stoss bleibt die neue Energieverteilung konstant, und die Gesamtenergie (E_{gesamt}) bleibt unverändert – was den Energieerhaltungssatz bestätigt.

(Siehe Abbildung 23)

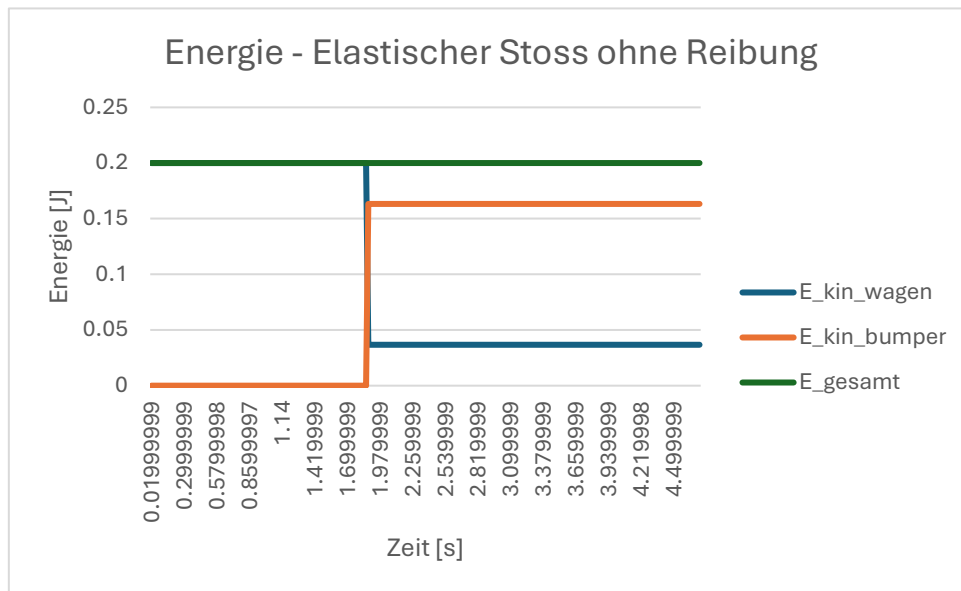


Abbildung 23: Plot Energie vom elastischer Stoss ohne Reibung

14.3.2. Energieverlauf – mit Reibung:

Erwartung:

Wird Reibung im System eingeführt, so handelt es sich nicht mehr um ein abgeschlossenes System. Reibung ist eine nicht-konservative Kraft, die mechanische Energie – insbesondere kinetische Energie – kontinuierlich in andere Energieformen wie Wärme umwandelt. Ausserdem wurde hier zusätzlich eine Druckfeder eingebaut, die temporär Energie speichert.

Erwartet wird daher:

- **$E_{\text{kin_wagen}}$ / $E_{\text{kin_bumper}}$:** nehmen mit der Zeit ab, da Energie durch Reibung verloren geht
- **E_{Feder} :** zeigt temporäre Anstiege bei Stössen (kurze Energieaufnahme)
- **E_{gesamt} :** sinkt schrittweise nach jedem Stoss → Energieerhaltung ist nicht gegeben, da Energie das System verlässt

Zudem sollte das System mit der Zeit zur Ruhe kommen – also gegen Null tendierende kinetische Energie.

Beobachtung:

Der Plot (Siehe Abbildung 24) bestätigt genau dieses Verhalten:

- Zu Beginn bewegt sich nur der Wagen mit einer kinetischen Energie von etwa 0.2 J.
- Beim Stoss mit dem Prellbock (z. B. bei $t \approx 30, 90, 230, 370$) wird ein Teil der Energie kurzfristig in der Feder gespeichert, bevor sie auf den Prellbock übertragen wird.
- Nach jedem Stoss ist die Gesamtenergie deutlich geringer als zuvor.
- Die kinetische Energie des Bumpers nimmt nach dem Stoss ebenfalls rasch ab – durch die Reibung auf seiner Bahn.
- Nach mehreren Interaktionen kommt das System schliesslich nahezu vollständig zur Ruhe (Energiesummen ≈ 0).

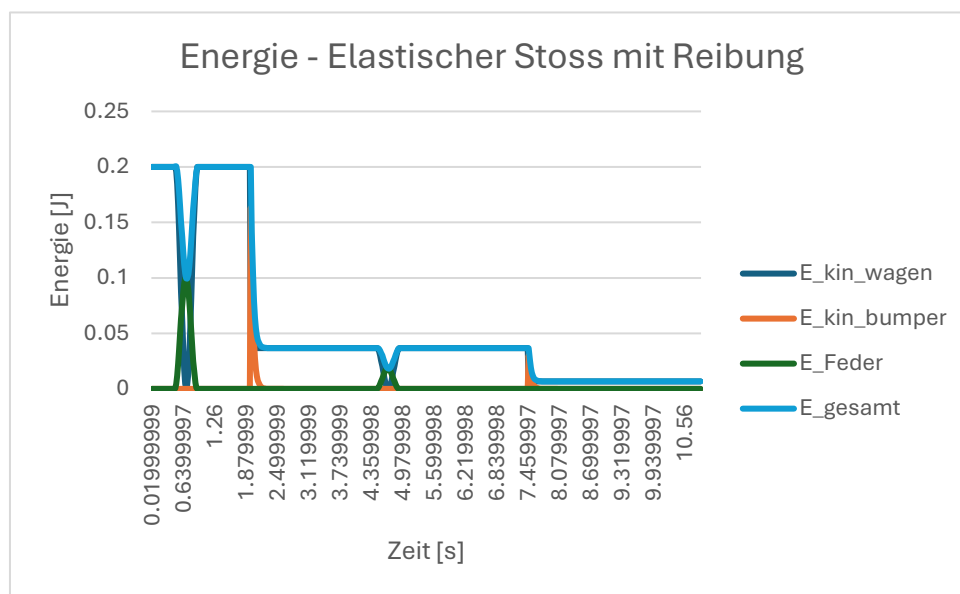


Abbildung 24: Plot Energie von elastischer Stoss mit Reibung

14.3.3. Fazit – Elastischer Stoss ohne und mit Reibung

Der Vergleich der beiden Szenarien zeigt deutlich die physikalischen Unterschiede zwischen einem idealisierten und einem realitätsnäheren Stossprozess:

- **Ohne Reibung** verhält sich das System ideal: Die gesamte kinetische Energie wird beim Stoss zwischen Wagen und Prellbock umverteilt. Die Gesamtenergie bleibt konstant, was die Gültigkeit des Energieerhaltungssatzes in einem abgeschlossenen, reibungsfreien System bestätigt. Es kommt zu keinem Energieverlust – die Bewegung kann theoretisch unendlich lange andauern.

- **Mit Reibung** hingegen tritt ein schrittweiser Energieverlust auf. Ein Teil der Bewegungsenergie wird über Reibungskräfte in Wärme umgewandelt und geht dem mechanischen System verloren. Die Feder wirkt dabei als temporärer Energiespeicher. Nach mehreren Stössen kommt das System allmählich zur Ruhe. Die Gesamtenergie nimmt über die Zeit kontinuierlich ab, was den Energieerhaltungssatz in offenen Systemen verletzt – jedoch physikalisch korrekt abbildet.

15. Zentraler inelastischer Stoss

15.1. Physikalische Beschreibung

Ein zentraler inelastischer Stoss ist ein Stoss zweier Körper, bei dem sie nach dem Zusammenstoss gemeinsam mit gleicher Geschwindigkeit weiterbewegen. Dabei bleibt der Impuls erhalten, aber mechanische Energie wird nicht vollständig übertragen – ein Teil geht durch Verformung oder Reibung verloren und wird z. B. in Wärme umgewandelt.

Impulserhaltung:

$$m_1 * v_1 + m_2 * v_2 = (m_1 + m_2) * v'$$

Energieverlust:

Die kinetische Energie vor dem Stoss ist grösser als danach:

$$E_{vor} > E_{nach}$$

$$\frac{1}{2} m_1 v_1 + \frac{1}{2} m_2 v_2 > \frac{1}{2} (m_1 + m_2) v'^2$$

Der Unterschied $\Delta E = E_{vor} - E_{nach}$ entspricht dem Energieverlust.

15.2. Beschreibung der Implementierung

Die Umsetzung erfolgte in Unity in einer zwei-dimensionalen Szene, bei der sich die Objekte reibungsfrei auf einem Luftkissentisch bewegen können.

Setup:

- Masse Wagen: 0.4 kg
- Masse Prellbock: 1.0 kg
- Die Szene wurde auf eine grosse Fläche erweitert, damit die Objekte nicht vom Tisch „fallen“.
- Die Bewegung war auf die z-Achse beschränkt.

Stossumsetzung:

- Beim Detektieren einer Kollision wurde der bisher verwendete elastische Stoß durch eine inelastische Verbindung mit einem FixedJoint ersetzt.
- Der FixedJoint verbindet den Wagen und den Prellbock genau zum Zeitpunkt des Stosses, sodass sie sich danach gemeinsam bewegen.
- Das ursprüngliche Federmodell wurde deaktiviert, um die Stossenergie nicht zu verfälschen.

```
// === inelastischer Stoß mit beweglichem linkem Bumper
float deltaLeft = transform.position.z - leftBumper.position.z - carWidth / 2 - bumperWidth / 2;
if (deltaLeft <= 0 && isLaunched && GetComponent<FixedJoint>() == null)
{
    // Joint hinzufügen, um die beiden Objekte untrennbar zu verbinden
    FixedJoint joint = gameObject.AddComponent<FixedJoint>();
    joint.connectedBody = leftBumper;
    joint.breakForce = Mathf.Infinity;
    joint.breakTorque = Mathf.Infinity;
    joint.enableCollision = false;

    Debug.Log("FixedJoint gesetzt - Inelastischer Stoß mit linkem Bumper");
}

// === right bumper ===

// === elastischer Stoß mit festem rechtem Bumper
if (transform.position.z >= rightBumper.position.z - carWidth / 2 - bumperWidth / 2)
{
    Vector3 vel = rb.linearVelocity;
    if (vel.z > 0) // nur wenn Auto auf Bumper zufährt
    {
        vel.z = -vel.z;
        rb.linearVelocity = vel;
        Debug.Log("Elastischer Stoß am rechten Bumper");
    }
}
```

15.3. Resultate und Interpretation

Für die Analyse des zentralen inelastischen Stosses wurden drei Zeitverläufe aufgezeichnet: Geschwindigkeit, Impuls und kinetische Energie. Die folgenden Abschnitte stellen jeweils die physikalische Erwartung und die Beobachtung aus der Simulation gegenüber.

1) Plot Geschwindigkeit – Zeit

Erwartung:

Vor dem Stoß bewegt sich nur der Wagen mit positiver Geschwindigkeit, während der Prellbock ruht. Zum Stoßzeitpunkt verbindet ein `FixedJoint` beide Körper, woraufhin sie sich gemeinsam mit einer reduzierten Geschwindigkeit bewegen. Diese Geschwindigkeit lässt sich mit dem Impulserhaltungsgesetz berechnen:

$$v' = \frac{m_1 * v_1 + m_2 * v_2}{m_1 + m_2}$$

Es wird erwartet, dass beide Körper nach dem Stoss dieselbe Geschwindigkeit haben und sich als ein System weiterbewegen.

Beobachtung:

Der Plot zeigt, dass sich der Wagen zu Beginn der Simulation mit konstanter positiver Geschwindigkeit (ca. 1 m/s) bewegt, während der Prellbock in Ruhe bleibt. Zum Stosszeitpunkt (ca. 0.9 s) ändert sich das Bewegungsverhalten abrupt:

- Die Geschwindigkeit des Wagens springt auf einen negativen Wert, was eine Richtungsumkehr bedeutet.
- Gleichzeitig beschleunigt der Prellbock kurzzeitig in die gleiche Richtung wie der Wagen, wird jedoch direkt durch die feste Verbindung mitgezogen.
- Nach der Verbindung (durch den FixedJoint) bewegen sich beide Körper mit konstanter gemeinsamer Geschwindigkeit.

Die klare Sprungstelle im Plot entspricht genau dem Stossereignis und zeigt eine sofortige Kopplung der Bewegungen. (Siehe Abbildung 25)

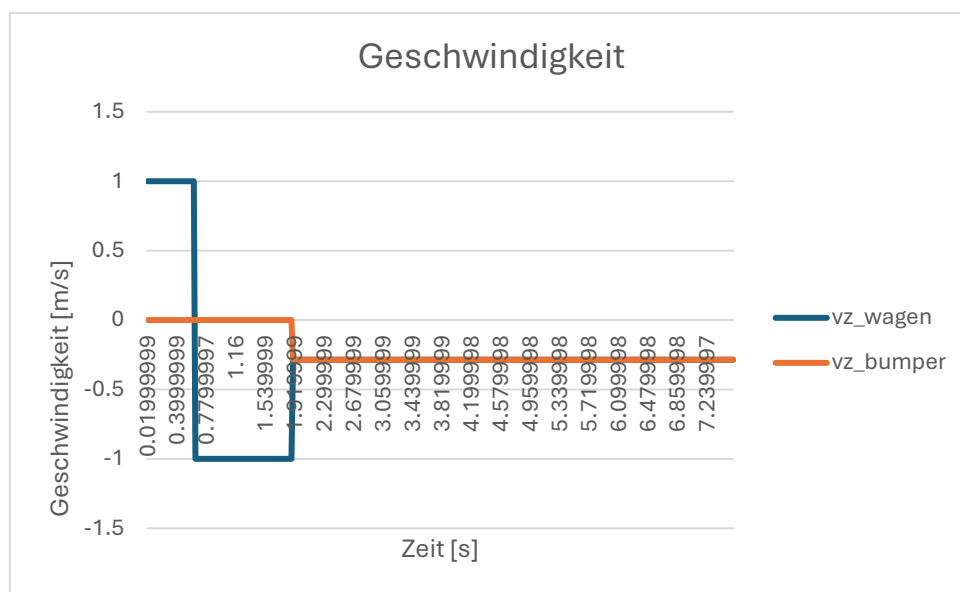


Abbildung 9: Plot der Geschwindigkeit vom zentralen inelastischen Stosses

2) Impuls – Zeit

Erwartung:

Da keine äusseren Kräfte wirken, sollte der **Gesamtimpuls** des Systems über die gesamte Zeit hinweg konstant bleiben. Vor dem Stoss trägt nur der Wagen den Impuls. Beim Stoss wird der Impuls auf den Prellbock übertragen, danach teilen sich beide Körper den Gesamtimpuls gleichmässig entsprechend ihrer Massen.

Beobachtung:

Zu Beginn trägt ausschliesslich der Wagen den Gesamtimpuls (ca. $+0.4 \text{ kg}\cdot\text{m/s}$), da der Prellbock ruht. Beim Stoss fällt der Impuls des Wagens stark ab (auf ca. $-0.28 \text{ kg}\cdot\text{m/s}$), während der des Prellbocks sprunghaft zunimmt. Auffällig ist (Siehe Abbildung 26):

- Die Summe der Impulse (p_{gesamt}) bleibt über den gesamten Zeitraum konstant bei ca. $0.4 \text{ kg}\cdot\text{m/s}$.
- Der Impulserhaltungssatz ist somit in der Simulation erfüllt.
- Nach dem Stoss sind die Einzelimpulse konstant und zeigen, dass beide Körper sich mit gemeinsamer Geschwindigkeit bewegen.

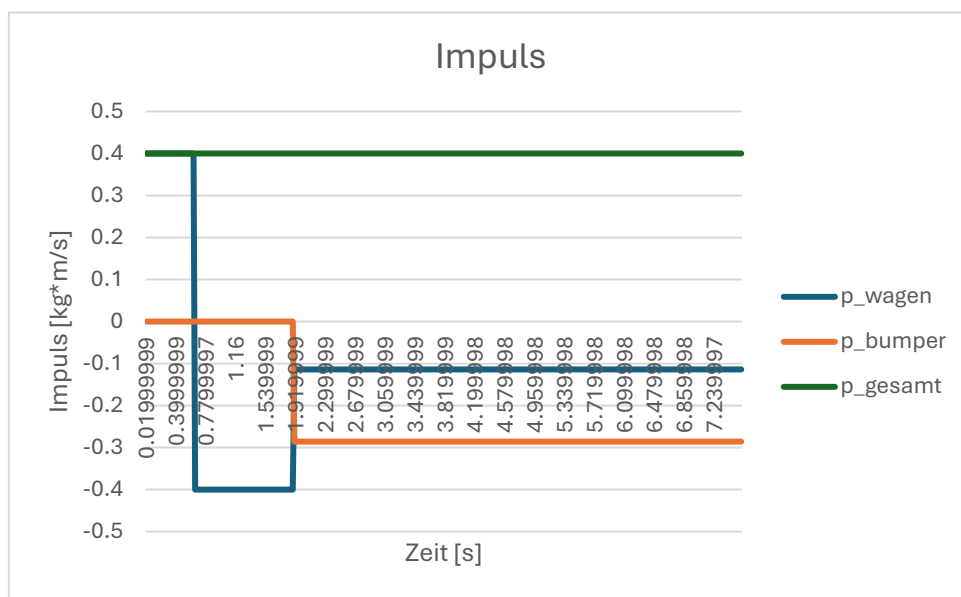


Abbildung 26: Plot des Impulses vom zentralen inelastischen Stosses

3) Kinetische Energie – Zeit

Erwartung:

Bei einem vollständig inelastischen Stoss geht ein Teil der kinetischen Energie verloren. Die Energie vor dem Stoss ist höher als danach. Es wird ein plötzlicher Abfall der Gesamtenergie zum Stosszeitpunkt erwartet. Die fehlende Energie wurde in der Realität z. B. durch innere Verformungen oder Wärme dissipiert.

Beobachtung:

Vor dem Stoss besitzt der Wagen nahezu die gesamte kinetische Energie ($\sim 0.2 \text{ s}$), während der Prellbock keine Bewegung und somit keine Energie trägt. Nach dem Stoss zeigt der Plot:

- Ein deutlicher Energieverlust beim Stosszeitpunkt: Die Gesamtenergie sinkt abrupt von ~ 0.2 s auf ~ 0.057 s.
- Die kinetischen Energien des Wagens und des Prellbocks teilen sich den Restbetrag realistisch auf.
- Es tritt kein weiterer Energieverlust nach dem Stoss auf, da keine Reibung modelliert wurde.

Diese plötzliche Reduktion der Gesamtenergie entspricht dem Energieverlust durch die inelastische Kopplung der beiden Körper – ganz im Sinne der Theorie. Die Summe der kinetischen Energien vor und nach dem Stoss bestätigt den berechneten Verlust. (Siehe Abbildung 27)

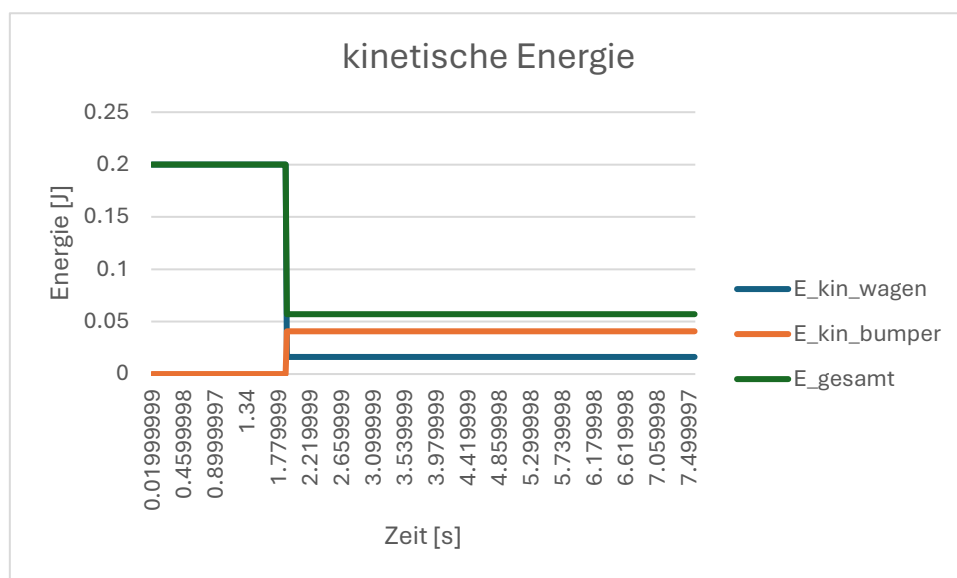


Abbildung 2710: Plot der kinetische Energie vom zentralen inelastischen Stosses

15.4. Fazit – Zentraler inelastischer Stoss

Die Simulation des zentralen inelastischen Stosses wurde erfolgreich mit Hilfe eines FixedJoint in Unity umgesetzt. Die beobachteten Zeitverläufe von Geschwindigkeit, Impuls und kinetischer Energie stimmen sehr gut mit den theoretischen Erwartungen überein:

- Die **Impulserhaltung** ist im System durchgängig gegeben – der Gesamtimpuls bleibt konstant, obwohl sich die Verteilung auf Wagen und Prellbock beim Stoss verändert.
- Die **kinetische Energie** hingegen ist – wie bei einem vollständig inelastischen Stoss zu erwarten – **nicht erhalten**. Ein signifikanter Anteil der ursprünglichen Bewegungsenergie wird beim Stoss in andere Energieformen überführt.

- Die **gemeinsame Bewegung** der beiden Körper nach dem Stoss mit identischer Geschwindigkeit bestätigt die vollständige Kopplung.

16. Inelastischer Stoss mit Rotation

16.1. Physikalische Beschreibung

In dieser Aufgabe wird ein nicht-zentraler inelastischer Stoss untersucht, bei dem der Wagen den Prellbock seitlich versetzt trifft. Dadurch entsteht ein Drehmoment am gemeinsamen Schwerpunkt des Systems, was zu einer Rotation der verbundenen Körper führt.

Drehimpulsübertragung beim exzentrischen Stoss

Bei einem inelastischen Stoß ohne äußere Drehmomente ist der Drehimpuls L bezüglich des Stoßpunkts erhalten:

$$L = r \times p = r \cdot m \cdot v$$

Der entstehende Drehimpuls L wird beim inelastischen Stoss auf die nun gekoppelte Struktur übertragen.

Dabei ist:

- r : der senkrechte Abstand zwischen der Stoßrichtung und dem Schwerpunkt der rotierenden Struktur,
- m : die Masse des Wagens,
- v : die Geschwindigkeit des Wagens zum Zeitpunkt des Stoßes.

Rotation nach dem Stoss

Nach der Kopplung bewegen sich Wagen und Prellbock als ein gemeinsames, zusammengesetztes System. Dieses rotiert um seinen gemeinsamen Massenmittelpunkt, wobei die Winkelgeschwindigkeit ω gemäß

$$\omega = \frac{L}{I}$$

aus dem übertragenen Drehimpuls L und dem Trägheitsmoment I berechnet werden kann.

Schwerpunkt der L-förmigen Struktur

Der gemeinsame Schwerpunkt $\mathbf{r}_s$ der verbundenen Körper wird mit der Massenmittelpunktformel berechnet:

$$\mathbf{r}_s = \frac{1}{\sum_{i=1}^n m_i} \sum_{i=1}^n m_i \mathbf{r}_i$$

Trägheitsmoment (Steiner'scher Satz)

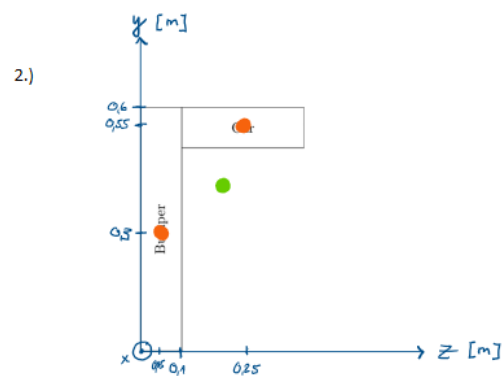
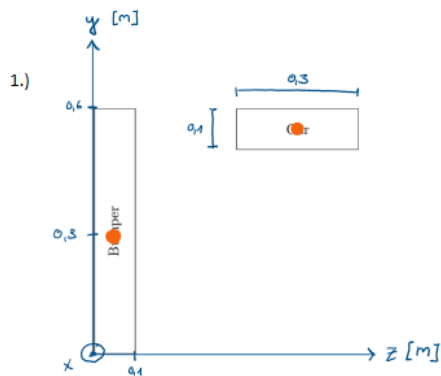
Das Trägheitsmoment der zusammengesetzten, L-förmigen Struktur wird mithilfe des **Satzes von Steiner** bestimmt. Dabei wird das Trägheitsmoment jedes Teilkörpers um seinen eigenen Schwerpunkt berechnet und mit einem Korrekturterm ergänzt:

$$I_{gesamt} = \sum_i (I_i + m_i d_i^2)$$

mit d_i = Abstand des Teilkörpers vom Gesamtschwerpunkt.

Berechnung des Schwerpunkts und des Trägheitsmoments

Zur Bestimmung des Trägheitsmoments der L-förmigen Struktur wurde zunächst der gemeinsame Schwerpunkt aus den Massen und Positionen von Wagen und Prellbock berechnet. Anschließend wurde das Trägheitsmoment I_{gesamt} der Struktur mit Hilfe des Steiner'schen Satzes bestimmt. Dazu wurden die Einzelträgheitsmomente um die jeweiligen Körperschwerpunkte berechnet und mit den Abständen zum Gesamtschwerpunkt korrigiert.



$$\vec{r}_{s,car} = \begin{pmatrix} 0 \\ 0,55m \\ 0,25m \end{pmatrix} \quad \vec{r}_{s,bumper} = \begin{pmatrix} 0 \\ 0,3m \\ 0,05m \end{pmatrix} \quad m_{car} = 0,4 \text{ kg} \\ m_{bumper} = 1 \text{ kg}$$

$$\begin{aligned} \vec{r}_{s,gesamt} &= \frac{1}{m_{car} + m_{bumper}} (m_{car} \cdot \vec{r}_{s,car} + m_{bumper} \cdot \vec{r}_{s,bumper}) \\ &= \frac{1}{0,4 + 1} \left(0,4 \cdot \begin{pmatrix} 0 \\ 0,55 \\ 0,25 \end{pmatrix} + 1 \cdot \begin{pmatrix} 0 \\ 0,3 \\ 0,05 \end{pmatrix} \right) \\ &= \frac{1}{1,4} \left(\begin{pmatrix} 0 \\ 0,22 \\ 0,1 \end{pmatrix} + \begin{pmatrix} 0 \\ 0,3 \\ 0,05 \end{pmatrix} \right) \\ &= \frac{1}{1,4} \begin{pmatrix} 0 \\ 0,52 \\ 0,15 \end{pmatrix} = \begin{pmatrix} 0 \\ \sim 0,3714m \\ \sim 0,1071m \end{pmatrix} \end{aligned}$$

$$I_{gesamt} = (I_{car} + m_{car} \cdot d_{car}^2) + (I_{bumper} + m_{bumper} \cdot d_{bumper}^2)$$

$$I_{car} = \frac{1}{12} m_{car} (a_{car}^2 + b_{car}^2)$$

$$I_{bumper} = \frac{1}{12} m_{bumper} (a_{bumper}^2 + b_{bumper}^2)$$

$$\begin{aligned} d_{car} &= \left| \vec{r}_{s,car} - \vec{r}_{s,gesamt} \right| = \left| \begin{pmatrix} 0 \\ 0,55 \\ 0,25 \end{pmatrix} - \begin{pmatrix} 0 \\ 0,3714 \\ 0,1071 \end{pmatrix} \right| \\ &= \left| \begin{pmatrix} 0 \\ 0,1786 \\ 0,1429 \end{pmatrix} \right| = \sqrt{0,1786^2 + 0,1429^2} \approx 0,2287m \end{aligned}$$

$$\begin{aligned} d_{bumper} &= \left| \vec{r}_{s,bumper} - \vec{r}_{s,gesamt} \right| = \left| \begin{pmatrix} 0 \\ 0,3 \\ 0,05 \end{pmatrix} - \begin{pmatrix} 0 \\ 0,3714 \\ 0,1071 \end{pmatrix} \right| \\ &= \left| \begin{pmatrix} 0 \\ -0,0714 \\ -0,0571 \end{pmatrix} \right| = \sqrt{0,0714^2 + 0,0571^2} \approx \underline{\underline{0,0914m}} \end{aligned}$$

$$I_{gesamt} = \frac{1}{12} 0,4 (0,1^2 + 0,3^2) + 0,4 \cdot 0,2287^2 + \frac{1}{12} 1 (0,1^2 + 0,6^2) + 1 \cdot 0,0914^2 \approx \underline{\underline{0,0634 \text{ kg} \cdot m^2}}$$

Energiebetrachtung

Da es sich um einen vollständig inelastischen Stoss handelt, ist die kinetische Gesamtenergie nach dem Stoss geringer als zuvor. Die Differenz wird als Energieverlust (z. B. durch Deformation, Wärme) interpretiert. Allerdings bleiben sowohl der lineare Impuls als auch der Drehimpuls im abgeschlossenen System erhalten.

16.2. Beschreibung der Implementierung

In Unity wurde das Setup wie folgt angepasst:

- Der Wagen wurde so versetzt, dass er den Prellbock **nicht mittig**, sondern exzentrisch trifft.

- Die vorherige Stosslogik wurde um die **Rotation nach dem Stoss** erweitert. Dazu wurden die RigidbodyConstraints des Systems wie folgt modifiziert:

```
rb.constraints = RigidbodyConstraints.FreezePositionX |
                RigidbodyConstraints.FreezePositionY |
                RigidbodyConstraints.FreezeRotationX |
                RigidbodyConstraints.FreezeRotationY;
```

→ Die Z-Rotation blieb **freigegeben**, um die Drehung zu ermöglichen.

16.3. Resultate und Interpretation

Winkelgeschwindigkeit und Drehbewegung

16.2. Beschreibung der Implementierung

Für die Simulation inelastischer Stöße mit Rotation wurde die bestehende Unity-Szene aus Teil 2 erweitert. Die Rollbahn wurde durch eine ebene Fläche ersetzt, die als Luftkissentisch dient, um Reibungseinflüsse zu minimieren. Die beteiligten Objekte – der Wagen (0.4 kg) und der L-förmige Prellbock (1.0 kg) – wurden so positioniert, dass der Wagen den Prellbock seitlich versetzt trifft.

Zur Umsetzung des inelastischen Stoßes wurde die zuvor verwendete Druckfeder entfernt und beim Kollisionereignis ein Fixed Joint dynamisch erzeugt. Dieses verbindet die beiden Körper dauerhaft miteinander, sodass sie sich nach dem Stoß wie ein starrer Körper gemeinsam bewegen. Die Kollisionserkennung erfolgte über Unitys OnCollisionEnter()-Callback, der das Fixed Joint zwischen Wagen und Prellbock erstellt.

Für die Rotation wurde die ursprünglich eingeschränkte Drehbarkeit des Prellbocks aufgehoben. Dazu wurden die Rigidbody-Constraints gezielt angepasst, sodass nur die Bewegung in der Ebene zugelassen war, während Drehungen um die z-Achse ermöglicht wurden:

```
rb.constraints = RigidbodyConstraints.FreezePositionY |
                RigidbodyConstraints.FreezeRotationX |
                RigidbodyConstraints.FreezeRotationZ;
```

Die Simulation erfasst kontinuierlich relevante physikalische Größen (Positionsdaten, lineare und Winkelgeschwindigkeit, Impuls, Energie) und speichert sie während der Laufzeit in einer CSV-Datei zur späteren Analyse. Die Auswertung dieser Daten erfolgte mit Python und ermöglichte eine direkte Gegenüberstellung von Simulationsergebnissen und theoretischen Erwartungen.

Die Implementierung wurde modular gehalten, sodass über den Unity Inspector verschiedene Szenarien (mit/ohne Reibung, elastisch/inelastisch, zentral/seitlich) leicht

konfigurierbar sind. Zusätzlich wurden Debug-Visualisierungen eingesetzt, um Stoßzeitpunkte, Schwerpunkte und Drehimpulsrichtungen sichtbar zu machen.

16.3. Resultate und Interpretation

Winkelgeschwindigkeit

Erwartung:

Durch den nicht-zentralen, inelastischen Stoß zwischen dem Wagen und dem seitlich versetzten Prellbock entsteht zusätzlich zur translatorischen Bewegung eine Rotationsbewegung. Aufgrund des Versatzes wirkt das Impulsübertragungsmoment nicht direkt durch den gemeinsamen Massenmittelpunkt, sondern erzeugt ein Drehmoment, das zum Rotieren der verbundenen Struktur führt. Erwartet wird eine sofort einsetzende konstante Winkelgeschwindigkeit ω nach der Kopplung, da keine äusseren Momente oder Reibungskräfte wirken.

Beobachtung:

Nach dem nicht-zentralen, inelastischen Stoß zeigt die Winkelgeschwindigkeit einen plötzlichen Abfall auf etwa -0.58 rad/s , gefolgt von einem konstanten Verlauf. Dies bestätigt, dass durch den exzentrischen Impuls ein Drehmoment erzeugt wurde, welches die L-förmige Struktur in Rotation versetzt. Die konstante ω nach dem Stoß weist darauf hin, dass keine äußeren Momente oder Reibungskräfte wirken und die Simulation das physikalische Verhalten korrekt abbildet. (Siehe Abbildung 28)

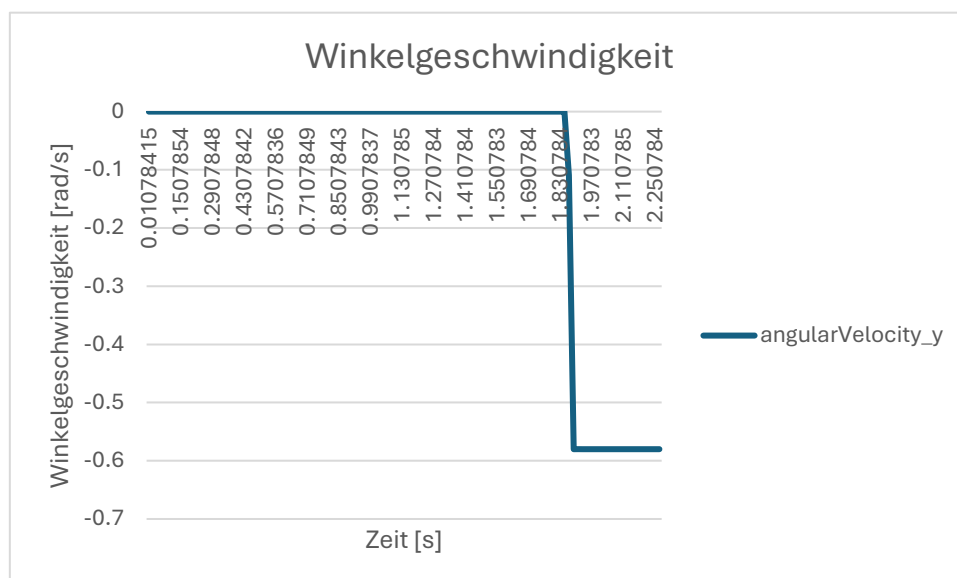


Abbildung 28: Plot der Winkelgeschwindigkeit vom inelastischen Stosses mit Rotation

Drehimpuls vor/nach dem Stoß

Erwartung:

Vor dem Stoß bewegt sich der Wagen geradlinig entlang der z-Achse und ist nicht mit dem Prellbock verbunden. Da er sich nicht um einen Punkt dreht und keine Rotation stattfindet, besitzt das System noch keinen Drehimpuls relativ zum späteren Rotationszentrum (dem gemeinsamen Schwerpunkt).

Erst beim exzentrischen, inelastischen Stoß wird der Wagen mit dem seitlich versetzten Prellbock gekoppelt. Dabei entsteht durch den Versatz ein Drehmoment, das zu einem plötzlichen Aufbau von Drehimpuls führt. Dieser Drehimpuls bleibt anschließend konstant erhalten, da keine äußeren Drehmomente auf das System wirken.

Beobachtung:

Zu Beginn der Bewegung ist der Drehimpuls null, da der Wagen sich geradlinig und ohne Rotation bewegt. Beim Stoß mit dem seitlich versetzten Prellbock tritt ein plötzlicher, negativer Sprung im Drehimpuls auf. Dieser markiert den Zeitpunkt, an dem durch den exzentrischen Kontakt ein Drehmoment entsteht und Drehimpuls auf die gekoppelte L-Struktur übertragen wird. Nach dem Stoß bleibt der Drehimpuls nahezu konstant, was bestätigt, dass keine äußeren Drehmomente wirken und der Drehimpuls im abgeschlossenen System erhalten bleibt. (Siehe Abbildung 29)

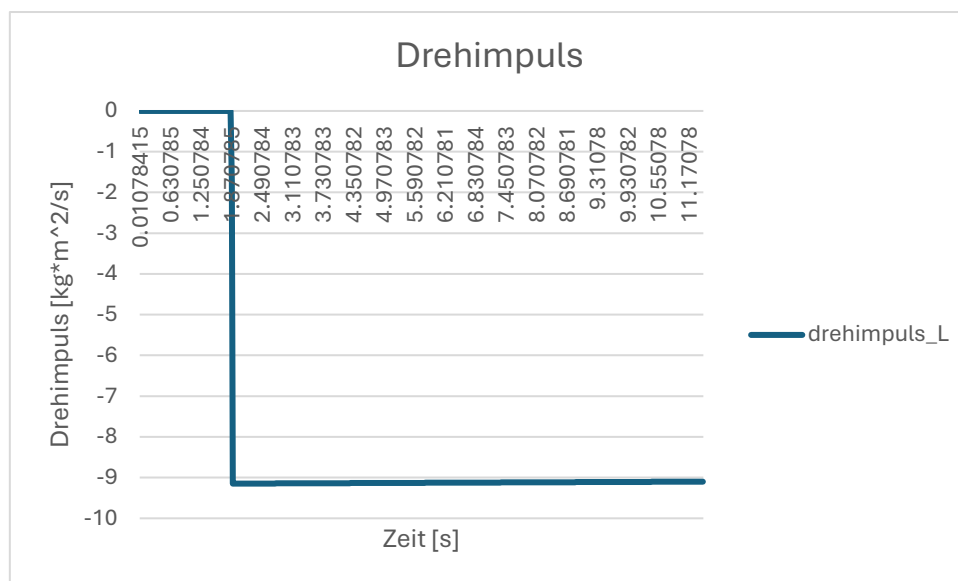


Abbildung 29: Plot des Drehimpulses vom inelastischen Stoßes mit Rotation

Kinetische Energie

Erwartung:

Da es sich um einen vollständig inelastischen Stoß handelt, ist ein **Energieverlust** unvermeidlich. Ein Teil der anfänglichen kinetischen Energie des Wagens wird in Rotationsenergie überführt, ein weiterer Teil geht durch irreversible Prozesse (z. B. Verformung oder Wärme) verloren. Die Gesamtenergie des Systems nach dem Stoß ist daher geringer:

$$\Delta E = E_{vor} - E_{nach} > 0$$

Beobachtung:

Zu Beginn der Bewegung besteht die kinetische Energie ausschließlich aus Translationsenergie des Wagens. Diese bleibt zunächst konstant. Beim Stoß mit dem seitlich versetzten Prellbock tritt ein plötzlicher Energieverlust auf, wie es für einen vollständig inelastischen Stoß typisch ist. Nach dem Stoß ist zusätzlich Rotationsenergie messbar, da sich die gekoppelte Struktur in Drehung versetzt. Die Gesamtenergie bleibt ab dem Stoßzeitpunkt auf einem niedrigeren Niveau weitgehend konstant, was auf das Fehlen weiterer Energieverluste hinweist. Die Differenz zur ursprünglichen Energie lässt sich auf irreversible Prozesse wie Wärme oder Verformung zurückführen. (Siehe Abbildung 30)

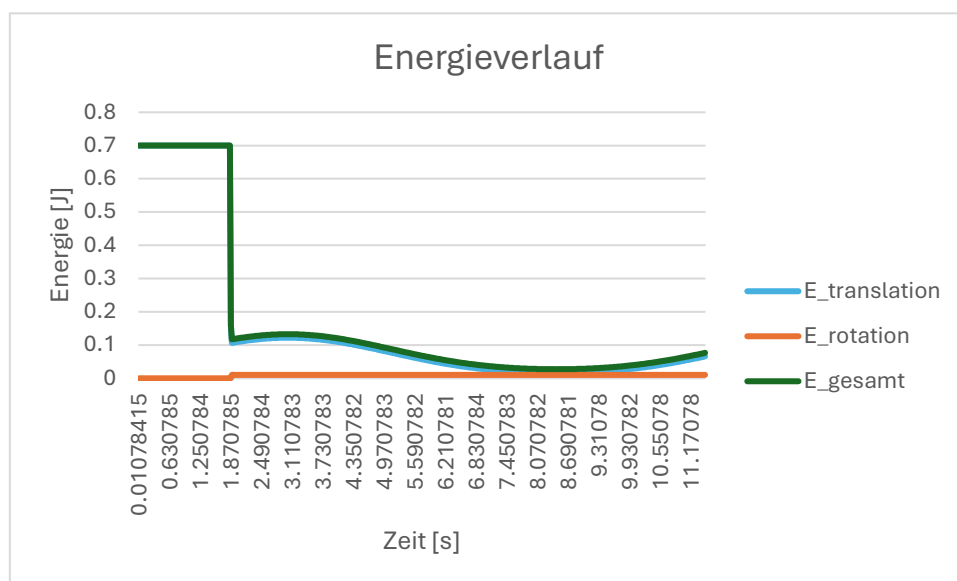


Abbildung 30: Plot des Energieverlaufs vom inelastischen Stoßes mit Rotation

1. Fazit zur Simulation inelastischer Stöße und Rotationen eines Wagens auf einer erweiterten Rollbahn mit Luftkissentisch

Die durchgeführte Simulation inelastischer Stöße auf einer erweiterten Rollbahn mit Luftkissentisch liefert ein realistisches und anschauliches Bild der physikalischen Vorgänge bei der Kopplung zweier Körper nach einem Stoß. Im Gegensatz zum idealisierten elastischen Fall wird bei einem inelastischen Stoß ein Teil der kinetischen Energie nicht erhalten, sondern in andere Energieformen umgewandelt – etwa in Wärme, Materialverformung oder intern gespeicherte Spannung durch die Verbindung der Körper.

Durch den versetzten Aufprall des Wagens auf den L-förmigen Prellbock entsteht zusätzlich ein Drehimpuls, der sich in eine Rotationsbewegung der verbundenen Struktur überträgt. Die Simulation zeigt, dass unter reibungsfreien und kraftfreien Bedingungen (idealisierte Luftkissentisch) sowohl der lineare Impuls als auch der Gesamtdrehimpuls erhalten bleiben – entsprechend den Gesetzen der Impulserhaltung.

Die Auswertung der Winkelgeschwindigkeit und der Impulsgrößen aus den Simulationsdaten liefert dabei Werte, die mit den theoretischen Berechnungen gut übereinstimmen. Kleinere Abweichungen lassen sich durch numerische Einflüsse, Zeitdiskretisierung oder interne Dämpfungsparameter in Unity erklären.

Gleichzeitig verdeutlicht das Szenario die Bedeutung des Angriffspunkts der Kraft relativ zum Schwerpunkt der Struktur: Nur ein versetzter Stoß kann überhaupt eine Rotation erzeugen. Die Koppelung der Körper durch ein Fixed Joint nach dem Stoß führt zu einer realistischen Nachbildung des zusammengesetzten Objekts, das sich translational und rotierend weiterbewegt.

Insgesamt bestätigt die Simulation die zentralen physikalischen Prinzipien und bietet eine wertvolle visuelle Unterstützung zum Verständnis von Impuls- und Energieumwandlung bei inelastischen Stößen mit Rotation. Zugleich zeigt sie aber auch die Grenzen numerischer Modelle auf, insbesondere wenn es um die exakte Energiebilanzierung oder ideale Reibungsfreiheit geht.

Anhang

```
// confine motion of the to 1D along z-axis
rb.constraints = RigidbodyConstraints.FreezePositionX | RigidbodyConstraints.FreezePositionY | RigidbodyConstraints.FreezeRotationX | RigidbodyConstraints.FreezeRotationY | RigidbodyConstraints.FreezeRotationZ;

// set motion of the bumpers
switch (bumperMode)
{
    case 0:
        //fix bumper in place
        leftBumper.constraints = RigidbodyConstraints.FreezeAll;
        break;
    case 1:
    case 2:
        //allow bumper to move along z-axis
        leftBumper.constraints = RigidbodyConstraints.FreezePositionX | RigidbodyConstraints.FreezePositionY |
                                RigidbodyConstraints.FreezeRotationX | RigidbodyConstraints.FreezeRotationY |
                                RigidbodyConstraints.FreezeRotationZ;
        break;
    default:
        Assert.Fail("Invalid bumper mode");
        break;
}

rightBumper.constraints = RigidbodyConstraints.FreezeAll;
```

Code 1: Bericht Teil 2, Bewegungseinschränkungen in Abhängigkeit zum Bumper Mode

```
float deltaLeft = transform.position.z - leftBumper.position.z - carWidth / 2 - bumperWidth / 2;
float deltaRight = transform.position.z - rightBumper.position.z + carWidth / 2 + bumperWidth / 2;

if (bumperMode == 0)
{
    // Left and right springs
    compressionLeft = springLength - deltaLeft;
    if (compressionLeft < 0)
    {
        compressionLeft = 0;
    }
    forceLeft = springConstant * compressionLeft;
    rb.AddForce(new Vector3(0f, 0f, forceLeft));

    compressionRight = springLength + deltaRight;
    if (compressionRight < 0)
    {
        compressionRight = 0;
    }
    forceRight = springConstant * compressionRight;
    rb.AddForce(new Vector3(0f, 0f, -forceRight));
}
else
{
    if (deltaLeft <= 0 && rb.linearVelocity.z < 0) // Collision detected
    {
        // Get masses and velocities
        float m1 = rb.mass; // Mass of the car
        float m2 = leftBumper.mass; // Mass of the left bumper
        float v1 = rb.linearVelocity.z; // Velocity of the car
        float v2 = leftBumper.linearVelocity.z; // Velocity of the left bumper

        // Calculate new velocities after elastic collision
        float v1New = ((m1 - m2) / (m1 + m2)) * v1;
        float v2New = ((m2 * m1) / (m1 + m2)) * v1;

        // Apply new velocities
        rb.linearVelocity = new Vector3(0f, 0f, v1New);
        leftBumper.linearVelocity = new Vector3(0f, 0f, v2New);
    }

    if (bumperMode == 2)
    {
        // friction force on the left bumper
        Vector3 frictionForce = -frictionCoefficient * leftBumper.linearVelocity.normalized * leftBumper.linearVelocity.magnitude;
        leftBumper.AddForce(frictionForce);
    }

    /// elastischer Stoß mit festem rechtem Bumper
    if (deltaRight >= 0)
    {
        Vector3 vel = rb.linearVelocity;
        if (vel.z > 0) // nur wenn Auto auf Bumper zufährt
        {
            vel.z = -vel.z;
            rb.linearVelocity = vel;
            Debug.Log("Elastischer Stoß am rechten Bumper");
        }
    }
}
```

Code 2: Bericht Teil 2, Stosslogik in Abhängigkeit zum Bumper Mode

```
// confine motion of the to 1D along z-axis
rb.constraints = RigidbodyConstraints.FreezePositionY |
               RigidbodyConstraints.FreezeRotationX |
               RigidbodyConstraints.FreezeRotationZ;

rightBumper.constraints = RigidbodyConstraints.FreezeAll;
```

Code 4: Bericht Teil 3, Bewegungseinschränkungen

```
if (hasHitLeftBumper && GetComponent<FixedJoint>() == null)
{
    FixedJoint joint = gameObject.AddComponent<FixedJoint>();
    joint.connectedBody = leftBumperCollision.rigidbody;
    joint.breakForce = Mathf.Infinity;
    joint.breakTorque = Mathf.Infinity;
    joint.enableCollision = true;
    joint.enablePreprocessing = false;

    Debug.Log("FixedJoint gesetzt - Inelastischer Stoß mit linkem Bumper");

    hasHitLeftBumper = false; // einmalige Ausführung
}
```

Code 3: Bericht Teil 3, Fixe Verbindung der beiden Elemente nach Kollision